The Concise TypeScript Book

Simone Poggiali

Özlü TypeScript Kitabı

Özlü TypeScript Kitabı, TypeScript'in yeteneklerine ilişkin kapsamlı ve kısa bir genel bakış sunar. Dilin güçlü tür sisteminden gelişmiş özelliklerine kadar en son sürümünün tüm yönlerini kapsayan açık açıklamalar içerir.

İster yeni başlayan ister deneyimli bir geliştirici olun, bu kitap TypeScript konusundaki anlayışınızı ve yetkinliğinizi geliştirmek için paha biçilmez bir kaynaktır.

Bu kitap tamamen ücretsiz ve açık kaynaklıdır.

Yüksek kaliteli teknik eğitimin herkes için erişilebilir olması gerektiğine inanıyorum. Bu nedenle kitabı ücretsiz olarak erişime açık tutuyor, iyileştirmeler ve yeni örneklerle düzenli olarak güncelliyorum.

The Concise TypeScript Book Plus Edition'ı keşfedin.



Açık kaynak sürümün ötesine geçmek isteyen okuyucular için **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7**, pratik uygulamalara odaklanan ek ve özel içerikler sunar.

Plus Edition şunları içerir:

- **TypeScript 7 için güncellendi** — en yeni TypeScript 7 özelliklerini ve dil iyileştirmelerini kapsar.
- **React ile TypeScript** — bileşenleri, props'ları, hook'ları, olayları, children'ı, ref'leri ve yaygın React kalıplarını türlendirmek için pratik rehberlik sunar.

- **Gerçek Dünya Projeleri için TypeScript Tarifleri** — geliştiricilerin TypeScript uygulamaları oluştururken ve sürdürürken karşılaştığı pratik sorunları ele alan odaklanmış örnekler sunar.

Plus Edition'ı satın alarak ücretsiz ve açık kaynaklı kitabın geliştirilmeye ve bakımının yapılmaya devam edilmesini de doğrudan desteklersiniz.

Plus Edition, dünya genelinde Amazon'da İngilizce ve İtalyanca olarak sunulmaktadır. Plus Edition'ı keşfedin ve Amazon'dan satın alın.

Projeyi Destekleyin

Ücretsiz kitap bir hatayı düzeltmenize, zor bir kavramı anlamanıza veya kariyerinizde ilerlemenize yardımcı olduysa, lütfen önerilen **$5** katkı payıyla istediğiniz tutarı ödeyerek ya da bir kahve ısmarlayarak çalışmamı desteklemeyi düşünün.

Desteğiniz, içeriği güncel tutmama ve yeni örnekler, daha açık açıklamalar ve ek pratik rehberlikle genişletmeme yardımcı olur.

BUY ME A COFFEE

Donate PayPal

Çeviriler

Bu kitap aşağıdakiler de dâhil olmak üzere birçok dile çevrilmiştir:

İngilizce

Bulgarca

Almanca

Fransızca

Endonezce

İtalyanca

Japonca

Korece

Lehçe

Portekizce (Brezilya)

İsveççe

Türkçe

Vietnamese

Çince

İspanyolca

Tayca

Rusça

Arapça Çekçe

İndirmeler ve web sitesi

EPUB sürümünü de indirebilirsiniz:

https://github.com/gibbok/typescript-book/tree/main/downloads

Çevrimiçi sürüme buradan ulaşabilirsiniz:

https://gibbok.github.io/typescript-book

İçindekiler

- Özlü TypeScript Kitabı
  - Projeyi Destekleyin
  - Çeviriler
  - İndirmeler ve web sitesi
  - İçindekiler
  - Giriş
  - Yazar Hakkında
  - TypeScript'e Giriş
    - TypeScript nedir?
    - Neden TypeScript?
    - TypeScript ve JavaScript
    - TypeScript Kod Üretimi
    - Günümüzde Modern JavaScript (Alt Sürüme Derleme)
  - TypeScript'e Başlarken
    - Kurulum
    - Yapılandırma
    - TypeScript Yapılandırma Dosyası
      - target
      - lib

        - strict
        - module
        - moduleResolution
        - esModuleInterop
        - jsx
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - TypeScript'e Geçiş Önerileri
- Tür Sistemini Keşfetme
    - TypeScript Dil Hizmeti
    - Yapısal Türleme
    - TypeScript'in Temel Karşılaştırma Kuralları
    - Kümeler Olarak Türler
    - Tür Atama: Tür Bildirimleri ve Tür Onaylamaları
        - Tür Bildirimi
        - Tür Onaylaması
        - Ortam Bildirimleri
    - Özellik Denetimi ve Fazla Özellik Denetimi
    - Zayıf Türler
    - Katı Nesne Değişmezi Denetimi (Tazelik)
    - Tür Çıkarımı
    - Daha Gelişmiş Çıkarımlar
    - Tür Genişletme
    - Const
        - Tür Parametrelerinde Const Değiştiricisi
        - Const Onaylaması
    - Açık Tür Ek Açıklaması
    - Tür Daraltma
        - Koşullar

        - Hata fırlatma veya dönüş
        - Ayırt Edici Birleşim
        - Kullanıcı Tanımlı Tür Korumaları
        - switch-true daraltması
    - İlkel Türler
        - string
        - boolean
        - number
        - bigint
        - Symbol
        - null ve undefined
        - Dizi
        - any
    - Tür Ek Açıklamaları
    - İsteğe Bağlı Özellikler
    - Salt Okunur Özellikler
    - Dizin İmzaları
    - Türleri Genişletme
    - Sabit Değer Türleri
    - Sabit Değer Çıkarımı
    - strictNullChecks
    - Enum'lar
        - Sayısal enum'lar
        - Dize enum'ları
        - Sabit enum'lar
        - Ters eşleme
        - Bildirimsel enum'lar
        - Hesaplanmış ve sabit üyeler
    - Daraltma
        - typeof tür korumaları
        - Doğruluk değeriyle daraltma
        - Eşitlikle daraltma

        - in Operatörüyle daraltma
        - instanceof ile daraltma
    - Atamalar
    - Kontrol Akışı Analizi
    - Tür Yüklemleri
    - Ayırt Edici Birleşimler
    - never Türü
    - Kapsamlılık denetimi
    - Nesne Türleri
    - Demet Türü (Anonim)
    - Adlandırılmış Demet Türü (Etiketli)
    - Sabit Uzunluklu Demet
    - Birleşim Türü
    - Kesişim Türleri
    - Tür Dizinleme
    - Değerden Tür
    - Fonksiyon Dönüşünden Tür
    - Modülden Tür
    - Eşlenmiş Türler
    - Eşlenmiş Tür Değiştiricileri
    - Koşullu Türler
    - Dağıtımlı Koşullu Türler
    - Koşullu Türlerde infer Tür Çıkarımı
    - Önceden Tanımlanmış Koşullu Türler
    - Şablon Birleşim Türleri
    - Any türü
    - Unknown türü
    - Void türü
    - Never türü
    - Arayüz ve Tür
        - Yaygın Sözdizimi
        - Temel Türler

        - Nesneler ve Arayüzler
        - Birleşim ve Kesişim Türleri
    - Yerleşik İlkel Türler
    - Yaygın Yerleşik JS Nesneleri
    - Aşırı Yüklemeler
    - Birleştirme ve Genişletme
    - Tür ve Arayüz Arasındaki Farklar
    - Sınıf
        - Sınıfın Yaygın Sözdizimi
        - Oluşturucu
        - Özel ve Korumalı Oluşturucular
        - Erişim Belirleyicileri
        - Get ve Set
        - Sınıflarda Otomatik Erişimciler
        - this
        - Parametre Özellikleri
        - Soyut Sınıflar
        - Jeneriklerle
        - Dekoratörler
            - Sınıf Dekoratörleri
            - Özellik Dekoratörü
            - Metot Dekoratörü
            - Getter ve Setter Dekoratörleri
            - Dekoratör Meta Verisi
        - Kalıtım
        - Statik Üyeler
        - Özellik Başlatma
        - Metot Aşırı Yükleme
    - Jenerikler
        - Jenerik Tür
        - Jenerik Sınıflar
        - Jenerik Kısıtlamalar

        - Jeneriklerde Bağlamsal Daraltma
    - Silinen Yapısal Türler
    - Ad Alanları
    - Semboller
    - Üç Eğik Çizgi Yönergeleri
    - Tür İşleme
        - Türlerden Tür Oluşturma
        - İndeksli Erişim Türleri
        - Yardımcı Türler
            - Awaited<T>
            - Partial<T>
            - Required<T>
            - Readonly<T>
            - Record<K, T>
            - Pick<T, K>
            - Omit<T, K>
            - Exclude<T, U>
            - Extract<T, U>
            - NonNullable<T>
            - Parameters<T>
            - ConstructorParameters<T>
            - ReturnType<T>
            - InstanceType<T>
            - ThisParameterType<T>
            - OmitThisParameter<T>
            - ThisType<T>
            - Uppercase<T>
            - Lowercase<T>
            - Capitalize<T>
            - Uncapitalize<T>
            - NoInfer<T>
    - Diğer Konular

- Hatalar ve İstisna İşleme
- Mixin Sınıfları
- Eşzamansız Dil Özellikleri
- Yineleyiciler ve Üreteçler
- TsDocs JSDoc Referansı
- @types
- JSX
- ES6 Modülleri
- ES7 Üs Alma Operatörü
- for-await-of İfadesi
- Yeni target Meta Özelliği
- Dinamik İçe Aktarma İfadeleri
- “tsc –watch”
- Null Olmayan Onaylama Operatörü
- Varsayılan Değerli Bildirimler
- İsteğe Bağlı Zincirleme
- Nullish Birleştirme Operatörü
- Şablon Değişmez Türleri
- İşlev Aşırı Yükleme
- Özyinelemeli Türler
- Özyinelemeli Koşullu Türler
- Node’da ECMAScript Modülü Desteği
- Onaylama Fonksiyonları
- Değişken Sayıda Öğeli Demet Türleri
- Kutulanmış Türler
- TypeScript’te Kovaryans ve Kontravaryans
    - Tür Parametreleri için İsteğe Bağlı Varyans Ek Açıklamaları
- Şablon Dizesi Kalıbı İndeks İmzaları
- satisfies Operatörü
- Yalnızca Tür İçe Aktarımları ve Dışa Aktarımı
- using Bildirimi ve Belirtik Kaynak Yönetimi

    - await using Bildirimi
- İçe Aktarma Nitelikleri
- Düzenli İfade Sözdizimi Denetimi
- import defer

Giriş

Özlü TypeScript Kitabı'na hoş geldiniz! Bu rehber, etkili TypeScript geliştirme için gerekli temel bilgileri ve pratik becerileri edinmenizi sağlar. Temiz ve sağlam kod yazmak için temel kavramları ve teknikleri keşfedin. İster yeni başlayan ister deneyimli bir geliştirici olun, bu kitap projelerinizde TypeScript'in gücünden yararlanmak için hem kapsamlı bir rehber hem de kullanışlı bir başvuru kaynağıdır.

Bu kitap TypeScript 7.0'ı kapsar.

Yazar Hakkında

Simone Poggiali, 90'lı yıllardan beri profesyonel kalitede kod yazma tutkusuna sahip deneyimli bir Staff Engineer'dır. Uluslararası kariyeri boyunca startup'lardan büyük kuruluşlara kadar çok çeşitli müşteriler için pek çok projeye katkıda bulunmuştur. HelloFresh, Siemens, O2, Leroy Merlin ve Snowplow gibi önemli şirketler onun uzmanlığından ve özverisinden yararlanmıştır.

Simone Poggiali'ye aşağıdaki platformlardan ulaşabilirsiniz:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social

- E-posta: gibbok.coding gmail.com

Katkıda bulunanların tam listesi: https://github.com/gibbok/typescript-book/graphs/contributors

TypeScript'e Giriş

TypeScript nedir?

TypeScript, JavaScript'i temel alan, güçlü tür sistemine sahip bir programlama dilidir. İlk olarak 2012'de Anders Hejlsberg tarafından tasarlanmıştır ve günümüzde Microsoft tarafından açık kaynaklı bir proje olarak geliştirilip sürdürülmektedir.

TypeScript, JavaScript'e derlenir ve herhangi bir JavaScript çalışma zamanında (örneğin bir tarayıcıda veya sunucudaki Node.js'de) çalıştırılabilir.

Fonksiyonel, jenerik, zorunlu ve nesne yönelimli programlama gibi birden fazla programlama paradigmasını destekler ve çalıştırılmadan önce JavaScript'e dönüştürülen, derlenen (transpile edilen) bir dildir.

Neden TypeScript?

TypeScript, yaygın programlama hatalarını önlemeye ve program çalıştırılmadan önce belirli türdeki çalışma zamanı hatalarından kaçınmaya yardımcı olan, güçlü tür sistemine sahip bir dildir.

Güçlü tür sistemine sahip bir dil, geliştiricinin veri türü tanımlarında çeşitli program kısıtlamalarını ve davranışlarını belirtmesine olanak tanıyarak yazılımın doğruluğunu

doğrulamayı ve kusurları önlemeyi kolaylaştırır. Bu, özellikle büyük ölçekli uygulamalarda değerlidir.

TypeScript'in bazı avantajları:

- İsteğe bağlı olarak güçlü tür sistemi sunan statik türleme
- Tür Çıkarımı
- ES6 ve ES7 özelliklerine erişim
- Platformlar ve tarayıcılar arası uyumluluk
- IntelliSense ile araç desteği

TypeScript ve JavaScript

TypeScript, `.ts` veya `.tsx` dosyalarına yazılırken JavaScript dosyaları `.js` veya `.jsx` uzantılarına sahiptir.

`.tsx` veya `.jsx` uzantılı dosyalar, React'te kullanıcı arayüzü geliştirmek için kullanılan JavaScript Sözdizimi Uzantısı JSX'i içerebilir.

TypeScript, sözdizimi açısından JavaScript'in (ECMAScript 2015) türlendirilmiş bir üst kümesidir. Tüm JavaScript kodları geçerli TypeScript kodudur, ancak bunun tersi her zaman doğru değildir.

Örneğin, aşağıdaki gibi `.js` uzantılı bir JavaScript dosyasındaki fonksiyonu ele alalım:

```
const sum = (a, b) => a + b;
```

Dosya uzantısı `.ts` olarak değiştirildiğinde fonksiyon dönüştürülerek TypeScript'te kullanılabilir. Ancak aynı fonksiyona TypeScript türleriyle ek açıklama eklenirse derlenmeden herhangi bir JavaScript çalışma zamanında

çalıştırılamaz. Aşağıdaki TypeScript kodu derlenmezse bir sözdizimi hatası üretir:

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript, geliştiricilerin tür ek açıklamaları aracılığıyla niyetlerini ifade etmelerine olanak tanıyarak olası çalışma zamanı hatalarını derleme zamanında tespit etmek üzere tasarlanmıştır. Ayrıca TypeScript, tür çıkarımı sayesinde açık tür ek açıklamaları verilmediğinde bile belirli sorunları yakalayabilir. Örneğin aşağıdaki kod parçacığında hiçbir TypeScript türü belirtilmez:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

Bu durumda TypeScript bir hata algılar ve şunu bildirir:

```
Property 'y' does not exist on type '{ x: number; }'.
```

TypeScript'in tür sistemi büyük ölçüde JavaScript'in çalışma zamanı davranışından etkilenir. Örneğin JavaScript'te dize birleştirme veya sayısal toplama yapabilen toplama operatörü (+), TypeScript'te de aynı şekilde modellenmiştir:

```
const result = '1' + 1; // Result is of type string
```

TypeScript'in arkasındaki ekip, JavaScript'in olağan dışı kullanımlarını bilinçli olarak hata şeklinde işaretlemeye karar vermiştir. Örneğin aşağıdaki geçerli JavaScript kodunu ele alalım:

```
const result = 1 + true; // In JavaScript, the result is equal
to 2
```

Ancak TypeScript bir hata verir:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Bu hata, TypeScript tür uyumluluğunu katı bir şekilde uyguladığı ve bu durumda bir sayı ile bir boole değeri arasında geçersiz bir işlem belirlediği için oluşur.

TypeScript Kod Üretimi

TypeScript derleyicisinin iki temel sorumluluğu vardır: tür hatalarını denetlemek ve JavaScript'e derlemek. Bu iki süreç birbirinden bağımsızdır. Türler derleme sırasında tamamen silindiğinden, kodun JavaScript çalışma zamanında yürütülmesini etkilemez. TypeScript, tür hataları bulunsa bile JavaScript çıktısı üretebilir. İşte tür hatası içeren bir TypeScript kodu örneği:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Argument of type 'string' is
not assignable to parameter of type 'number'.
```

Ancak yine de çalıştırılabilir JavaScript çıktısı üretebilir:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

TypeScript türlerini çalışma zamanında denetlemek mümkün değildir. Örneğin:

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
```

```
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' only refers to a type, but is being used as a
value here.
        // ...
    }
};
```

Türler derlemeden sonra silindiği için bu kodu JavaScript'te çalıştırmanın bir yolu yoktur. Türleri çalışma zamanında tanımak için başka bir mekanizma kullanmamız gerekir. TypeScript birçok seçenek sunar; bunların yaygın olanlarından biri "etiketli birleşim"dir. Örneğin:

```
interface Dog {
    kind: 'dog'; // Tagged union
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // Tagged union
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
```

```
};
makeNoise(dog);
```

“kind” özelliği, JavaScript’te nesneleri birbirinden ayırmak için çalışma zamanında kullanılabilen bir değerdir.

Çalışma zamanındaki bir değerin, tür bildiriminde belirtilenden farklı bir türe sahip olması da mümkündür. Örneğin geliştirici bir API türünü yanlış yorumlayıp ona hatalı bir ek açıklama eklemiş olabilir.

TypeScript, JavaScript’in bir üst kümesi olduğundan “class” anahtar sözcüğü çalışma zamanında hem tür hem de değer olarak kullanılabilir.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
```

```
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

JavaScript'te bir "class", "prototype" özelliğine sahiptir ve "instanceof" operatörü, bir oluşturucunun prototype özelliğinin bir nesnenin prototip zincirinin herhangi bir yerinde bulunup bulunmadığını sınamak için kullanılabilir.

Tüm türler silindiğinden TypeScript'in çalışma zamanı performansı üzerinde hiçbir etkisi yoktur. Ancak TypeScript, derleme süresine bir miktar ek yük getirir.

Günümüzde Modern JavaScript (Alt Sürüme Derleme)

TypeScript, kodu ECMAScript 3'ten (1999) bu yana yayımlanmış herhangi bir JavaScript sürümüne derleyebilir. Bu, TypeScript'in en yeni JavaScript özelliklerini eski sürümlere transpile edebileceği anlamına gelir; bu süreç Downleveling olarak bilinir. Böylece eski çalışma zamanı ortamlarıyla en yüksek uyumluluk korunurken modern JavaScript kullanılabilir.

JavaScript'in eski bir sürümüne transpile edilirken TypeScript'in, yerel uygulamalara kıyasla performans ek yükü oluşturabilecek kodlar üretebileceğini unutmamak önemlidir.

TypeScript'te kullanılabilecek modern JavaScript özelliklerinden bazıları şunlardır:

- AMD tarzı "define" geri çağrıları veya CommonJS "require" deyimleri yerine ECMAScript modülleri.
- Prototipler yerine sınıflar.
- "var" yerine "let" veya "const" kullanılarak değişken bildirimi.
- Geleneksel "for" döngüsü yerine "for-of" döngüsü veya ".forEach".
- Fonksiyon ifadeleri yerine ok fonksiyonları.
- Yapıyı ayırma ataması.
- Kısaltılmış özellik/metot adları ve hesaplanmış özellik adları.
- Varsayılan fonksiyon parametreleri.

Geliştiriciler bu modern JavaScript özelliklerinden yararlanarak TypeScript'te daha anlamlı ve kısa kodlar yazabilir.

TypeScript'e Başlarken

Kurulum

Visual Studio Code, TypeScript dili için mükemmel destek sunar, ancak TypeScript derleyicisini içermez. TypeScript derleyicisini kurmak için npm veya yarn gibi bir paket yöneticisi kullanabilirsiniz:

```
npm install typescript --save-dev
```

veya

```
yarn add typescript --dev
```

Her ekip üyesinin aynı TypeScript sürümünü kullanmasını sağlamak için oluşturulan kilit dosyasını kaynak denetimine eklediğinizden emin olun.

TypeScript derleyicisini çalıştırmak için aşağıdaki komutları kullanabilirsiniz.

```
npx tsc
```

veya

```
yarn tsc
```

Daha öngörülebilir bir derleme süreci sağladığından TypeScript'i global olarak değil, proje bazında kurmanız önerilir. Ancak tek seferlik durumlarda aşağıdaki komutu kullanabilirsiniz:

```
npx tsc
```

veya global olarak kurabilirsiniz:

```
npm install -g typescript
```

Microsoft Visual Studio kullanıyorsanız MSBuild projeleriniz için TypeScript'i NuGet'te bir paket olarak edinebilirsiniz. NuGet Paket Yöneticisi Konsolu'nda aşağıdaki komutu çalıştırın:

```
Install-Package Microsoft.TypeScript.MSBuild
```

TypeScript kurulumu sırasında iki yürütülebilir dosya kurulur: TypeScript derleyicisi olarak "tsc" ve bağımsız TypeScript sunucusu olarak "tsserver". Bağımsız sunucu, akıllı kod

tamamlama sağlamak için editörler ve IDE'ler tarafından kullanılabilen derleyiciyi ve dil hizmetlerini içerir.

Ayrıca Babel (bir eklenti aracılığıyla) veya swc gibi TypeScript uyumlu çeşitli transpiler'lar bulunmaktadır. Bu transpiler'lar, TypeScript kodunu başka hedef dillere veya sürümlere dönüştürmek için kullanılabilir.

TypeScript 7.0, derleyicinin ve dil hizmetinin yerel bir uygulaması olarak Go ile yeniden yazıldı. Tam derlemeleri ve editör özelliklerini hızlandırmak için paylaşımlı bellekli çoklu iş parçacığı ve diğer optimizasyonları kullanarak geliştirme sırasındaki geri bildirim süresini kısaltır.

Bazı TypeScript 7.0 performans özellikleri ayarlanabilir. Tür denetimi `--checkers` ile paralel işçilerde çalışabilir; daha fazla işçi büyük projeleri hızlandırabilir, ancak daha fazla bellek kullanır. Yeniden oluşturulan `--watch` modu, platformlar arası dosya izlemeyi iyileştirir. TypeScript 7.0 henüz (Temmuz 2026 itibarıyla) bir derleyici API'si içermez; bu nedenle hâlâ TypeScript 6.0 API'sine ihtiyaç duyan araçlar, `@typescript/typescript6` veya npm alias'ları kullanılarak TypeScript 7.0 ile yan yana çalıştırılabilir.

Yapılandırma

TypeScript, tsc CLI seçenekleri kullanılarak veya projenin kök dizinine yerleştirilen tsconfig.json adlı özel bir yapılandırma dosyasından yararlanılarak yapılandırılabilir.

Önerilen ayarların önceden doldurulduğu bir tsconfig.json dosyası oluşturmak için aşağıdaki komutu kullanabilirsiniz:

```
tsc --init
```

`tsc` komutu yerel olarak çalıştırıldığında TypeScript, kodu en yakın tsconfig.json dosyasında belirtilen yapılandırmayı kullanarak derler.

Varsayılan ayarlarla çalışan bazı CLI komutu örnekleri şunlardır:

```
tsc main.ts // Compile a specific file (main.ts) to JavaScript
tsc src/*.ts // Compile any .ts files under the 'src' folder to JavaScript
tsc app.ts util.ts --outfile index.js // Compile two TypeScript files (app.ts and util.ts) into a single JavaScript file (index.js)
```

TypeScript Yapılandırma Dosyası

TypeScript Derleyicisini (tsc) yapılandırmak için bir tsconfig.json dosyası kullanılır. Bu dosya genellikle `package.json` dosyasıyla birlikte projenin kök dizinine eklenir.

Notlar:

- tsconfig.json, json biçiminde olmasına rağmen yorumları kabul eder.
- Komut satırı seçenekleri yerine bu yapılandırma dosyasını kullanmanız önerilir.

Aşağıdaki bağlantıda tüm belgeleri ve bunların şemasını bulabilirsiniz:

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

Aşağıda yaygın ve kullanışlı yapılandırmaların bir listesi yer almaktadır:

target

“target” özelliği, TypeScript kodunuzun hangi ECMAScript sürümüne çıktı vermesi/derlenmesi gerektiğini belirtmek için kullanılır. Modern tarayıcılar için ES6 iyi bir seçenektir. Not: ES5 desteği TypeScript 6.0’da kullanımdan kaldırılmıştır ve TypeScript 7.0’da artık desteklenmemektedir.

lib

“lib” özelliği, derleme sırasında hangi kütüphane dosyalarının dâhil edileceğini belirtmek için kullanılır. TypeScript, “target” özelliğinde belirtilen özelliklere yönelik API’leri otomatik olarak dâhil eder, ancak belirli ihtiyaçlar için belirli kütüphaneleri hariç tutmak veya seçmek mümkündür. Örneğin bir sunucu projesi üzerinde çalışıyorsanız yalnızca tarayıcı ortamında kullanışlı olan “DOM” kütüphanesini hariç tutabilirsiniz.

strict

“strict” seçeneği daha güçlü denetimleri etkinleştirerek tür güvenliğini iyileştirir. TypeScript 6.0’dan itibaren varsayılan olarak etkindir; aksi durumda tsconfig.json dosyanızda açıkça true olarak ayarlamanız gerekir. “strict” seçeneğini etkinleştirmek TypeScript’in şunları yapmasını sağlar:

- Her kaynak dosya için “use strict” kullanan kod üretir.

- Tür denetimi sürecinde “null” ve “undefined” değerlerini dikkate alır.
- Tür ek açıklamaları olmadığında “any” türünün kullanımını devre dışı bırakır.
- Aksi takdirde “any” türünü ifade edecek olan “this” ifadesinin kullanımında hata verir.

module

“module” özelliği, derlenen program için desteklenen modül sistemini ayarlar. Çalışma zamanında, belirtilen modül sistemine göre bağımlılıkları bulmak ve yürütmek için bir modül yükleyici kullanılır.

JavaScript’te en yaygın kullanılan modül yükleyiciler, sunucu tarafı uygulamaları için Node.js CommonJS ve tarayıcı tabanlı web uygulamalarındaki AMD modülleri için RequireJS’dir. TypeScript; UMD, System, ESNext, ES2015/ES6 ve ES2020 dâhil olmak üzere çeşitli modül sistemleri için kod üretebilir. Modül sistemi, hedef ortama ve o ortamda kullanılabilen modül yükleme mekanizmasına göre seçilmelidir.

Not: Eski modül sistemleri (AMD, UMD, SystemJS) desteği TypeScript 6.0’da kullanımdan kaldırılmıştır ve TypeScript 7.0’da artık desteklenmemektedir.

moduleResolution

“moduleResolution” özelliği, modül çözümleme stratejisini belirtir. Modern TypeScript kodu için “nodenext” veya “bundler” kullanın. “classic” stratejisi yalnızca eski TypeScript sürümleri (1.6’dan önceki sürümler) için kullanılır.

esModuleInterop

“esModuleInterop” özelliği, “default” özelliğini kullanarak dışa aktarım yapmayan CommonJS modüllerinden varsayılan içe aktarmalara izin verir; bu özellik, üretilen JavaScript’te uyumluluk sağlamak için bir shim sunar. Bu seçeneği etkinleştirdikten sonra `import * as MyLibrary from "my-library"` yerine `import MyLibrary from "my-library"` kullanabiliriz.

“esModuleInterop”, bozucu değişikliklerden kaçınmak için başlangıçta isteğe bağlıydı, ancak uzun süredir önerilen varsayılan ayardır. Devre dışı bırakılması, CommonJS ile ESM kullanılırken belli belirsiz çalışma zamanı sorunlarına yol açabilir. Not: TypeScript 6.0’dan itibaren bu daha güvenli birlikte çalışabilirlik davranışı her zaman etkindir.

TypeScript 6.0’da bazı eski yapılandırma seçenekleri ve sözdizimi biçimleri kullanımdan kaldırılmış veya eski davranış üzerinden geçiş yapmıştır. TypeScript 7.0’da bunlar kesin hatalara veya işlem yapmayan davranışlara dönüşmüştür.

İşlem yapmayan davranışla birlikte kesin hatalara dönüşen kullanımdan kaldırmalar şunlardır:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- `esModuleInterop` veya `allowSyntheticDefaultImports` seçeneğinin devre dışı bırakılması
- `alwaysStrict` seçeneğinin devre dışı bırakılması

- ad alanı bildirimlerinde `module` anahtar sözcüğü
- içe aktarmalarda `asserts`
- `/// <reference no-default-lib />`, `skipDefaultLibCheck` kapsamında
- yerel bir `tsconfig.json` ile CLI dosya yolları (`--ignoreConfig` kullanılmadığı sürece)

jsx

“jsx” özelliği yalnızca ReactJS’de kullanılan .tsx dosyalarına uygulanır ve JSX yapılarının JavaScript’e nasıl derleneceğini kontrol eder. Yaygın bir seçenek olan “preserve”, JSX’i değiştirmeden koruyarak bir .jsx dosyasına derler; böylece daha sonraki dönüşümler için Babel gibi farklı araçlara aktarılabilir.

skipLibCheck

“skipLibCheck” özelliği, TypeScript’in içe aktarılan üçüncü taraf paketlerin tamamında tür denetimi yapmasını önler. Bu özellik, bir projenin derleme süresini azaltır. TypeScript, kodunuzu bu paketler tarafından sağlanan tür tanımlarına göre denetlemeye devam eder.

files

“files” özelliği, programa her zaman dahil edilmesi gereken dosyaların listesini derleyiciye bildirir.

include

“include” özelliği, dahil etmek istediğimiz dosyaların listesini derleyiciye bildirir. Bu özellik, herhangi bir alt dizin için “*”, *herhangi bir dosya adı için* “” ve isteğe bağlı karakterler için “?” gibi glob benzeri kalıplara izin verir.

exclude

“exclude” özelliği, derlemeye dahil edilmemesi gereken dosyaların listesini belirtir. Bunlar “node_modules” veya test dosyaları gibi dosyaları içerebilir. Not: tsconfig.json yorumlara izin verir.

importHelpers

TypeScript, belirli gelişmiş veya daha eski JavaScript sürümlerine indirgenen özellikler için kod oluştururken yardımcı kod kullanır. Varsayılan olarak bu yardımcılar, onları kullanan dosyalarda yinelenir. `importHelpers` seçeneği bunun yerine bu yardımcıları `tslib` modülünden içe aktararak JavaScript çıktısını daha verimli hâle getirir.

TypeScript’e Geçiş Önerileri

Büyük projelerde, TypeScript ve JavaScript kodunun başlangıçta birlikte bulunacağı kademeli bir geçişin benimsenmesi önerilir. Yalnızca küçük projeler tek seferde TypeScript’e geçirilebilir.

Bu geçişin ilk adımı, TypeScript’i derleme zinciri sürecine dahil etmektir. Bu, .ts ve .tsx dosyalarının mevcut JavaScript dosyalarıyla birlikte bulunmasına izin veren “allowJs” derleyici seçeneği kullanılarak yapılabilir. TypeScript, JavaScript

dosyalarından türü çıkaramadığında bir değişken için “any” türüne geri döneceğinden, geçişin başında derleyici seçeneklerinizde “noImplicitAny” seçeneğini devre dışı bırakmanız önerilir.

İkinci adım, her modülü dönüştürürken testleri çalıştırabilmeniz için JavaScript testlerinizin TypeScript dosyalarıyla birlikte çalışmasını sağlamaktır. Jest kullanıyorsanız TypeScript projelerini Jest ile test etmenizi sağlayan `ts-jest` aracını kullanmayı değerlendirin.

Üçüncü adım, üçüncü taraf kütüphaneler için tür bildirimlerini projenize dahil etmektir. Bu bildirimler paketle birlikte veya DefinitelyTyped üzerinde bulunabilir. Bunları https://www.typescriptlang.org/dt/search adresinden arayabilir ve şu komutla yükleyebilirsiniz:

```
npm install --save-dev @types/package-name
```

veya

```
yarn add --dev @types/package-name
```

Dördüncü adım, yapraklardan başlayarak bağımlılık grafiğinizi izleyip aşağıdan yukarıya bir yaklaşımla modül modül geçiş yapmaktır. Buradaki fikir, başka modüllere bağımlı olmayan modülleri dönüştürmeye başlamaktır. Bağımlılık grafiklerini görselleştirmek için “madge” aracını kullanabilirsiniz.

Yardımcı işlevler ile harici API’ler veya belirtimlerle ilgili kodlar, bu ilk dönüşümler için iyi aday modüllerdir. Swagger sözleşmelerinden, GraphQL veya JSON şemalarından projenize dahil edilecek TypeScript tür tanımlarını otomatik olarak oluşturmak mümkündür.

Herhangi bir belirtim veya resmî şema bulunmadığında, bir sunucunun döndürdüğü JSON gibi ham verilerden türler oluşturabilirsiniz. Ancak uç durumların gözden kaçmasını önlemek için türlerin veriler yerine belirtimlerden oluşturulması önerilir.

Geçiş sırasında kodu yeniden düzenlemekten kaçının ve yalnızca modüllerinize tür eklemeye odaklanın.

Beşinci adım, tüm türlerin bilinmesini ve tanımlanmasını zorunlu kılacak olan “noImplicitAny” seçeneğini etkinleştirerek projeniz için daha iyi bir TypeScript deneyimi sağlamaktır.

Geçiş sırasında, bir JavaScript dosyasında TypeScript tür denetimini etkinleştiren `@ts-check` yönergesini kullanabilirsiniz. Bu yönerge, tür denetiminin esnek bir sürümünü sağlar ve başlangıçta JavaScript dosyalarındaki sorunları belirlemek için kullanılabilir. Bir dosyaya `@ts-check` eklendiğinde TypeScript, JSDoc biçimindeki yorumları kullanarak tanımları çıkarmaya çalışır. Ancak JSDoc ek açıklamalarını yalnızca geçişin çok erken bir aşamasında kullanmayı değerlendirin.

tsconfig.json dosyanızda `noEmitOnError` seçeneğinin varsayılan değeri olan false değerini korumayı değerlendirin. Bu, hatalar bildirilse bile JavaScript kaynak kodu çıktısı almanıza olanak tanır.

Tür Sistemini Keşfetme

TypeScript Dil Hizmeti

tsserver olarak da bilinen TypeScript Dil Hizmeti; hata bildirme, tanılama, kaydetme sırasında derleme, yeniden

adlandırma, tanıma gitme, tamamlama listeleri, imza yardımı ve daha fazlası gibi çeşitli özellikler sunar. Öncelikle tümleşik geliştirme ortamları (IDE'ler) tarafından IntelliSense desteği sağlamak için kullanılır. Visual Studio Code ile sorunsuz biçimde tümleşir ve Conquer of Completion (Coc) gibi araçlar tarafından kullanılır.

Geliştiriciler, TypeScript düzenleme deneyimini geliştirmek için özel bir API'den yararlanabilir ve kendi özel dil hizmeti eklentilerini oluşturabilir. Bu, özellikle özel lint özelliklerini uygulamak veya özel bir şablon dili için otomatik tamamlamayı etkinleştirmek açısından yararlı olabilir.

Gerçek dünyadan bir özel eklenti örneği, styled components içindeki CSS özellikleri için sözdizimi hata bildirimi ve IntelliSense desteği sağlayan "typescript-styled-plugin" eklentisidir.

Daha fazla bilgi ve hızlı başlangıç kılavuzları için GitHub'daki resmî TypeScript Wiki'sine başvurabilirsiniz: https://github.com/microsoft/TypeScript/wiki/

Yapısal Türleme

TypeScript, yapısal bir tür sistemini temel alır. Bu, türlerin uyumluluğunun ve eşdeğerliğinin C# veya C gibi nominal tür sistemlerinde olduğu gibi adları ya da bildirim yerleri yerine gerçek yapıları veya tanımları tarafından belirlendiği anlamına gelir.

TypeScript'in yapısal tür sistemi, JavaScript'in dinamik duck typing sisteminin çalışma zamanındaki işleyişi temel alınarak tasarlanmıştır.

Aşağıdaki örnek geçerli TypeScript kodudur. Gözlemleyebileceğiniz gibi “X” ve “Y”, farklı bildirim adlarına sahip olsalar da aynı “a” üyesine sahiptir. Türler yapılarına göre belirlenir ve bu durumda yapılar aynı olduğundan uyumlu ve geçerlidir.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Valid
```

TypeScript’in Temel Karşılaştırma Kuralları

TypeScript’in karşılaştırma süreci özyinelemelidir ve herhangi bir düzeyde iç içe yerleştirilmiş türler üzerinde yürütülür.

“Y”, en az “X” ile aynı üyelere sahipse “X” türü “Y” ile uyumludur.

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // Valid, as it has at least the
same members as X
const r: X = y;
```

İşlev parametreleri adlarına göre değil, türlerine göre karşılaştırılır:

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
```

```
y = x; // Valid
x = y; // Valid
```

İşlev dönüş türleri aynı olmalıdır:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Invalid
x = y; // Invalid
```

Bir kaynak işlevin dönüş türü, hedef işlevin dönüş türünün bir alt türü olmalıdır:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Valid
y = x; // Invalid member b is missing
```

İşlev parametrelerinin yok sayılmasına JavaScript'te yaygın bir uygulama olduğundan izin verilir; örneğin "Array.prototype.map()" kullanımı:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Bu nedenle aşağıdaki tür bildirimleri tamamen geçerlidir:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Missing b parameter
y = x; // Valid
```

Kaynak türün tüm ek isteğe bağlı parametreleri geçerlidir:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
```

```
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; //Valid
```

Hedef türün kaynak türde karşılığı olmayan tüm isteğe bağlı parametreleri geçerlidir ve hata oluşturmaz:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; // Valid
```

Rest parametresi, sonsuz bir isteğe bağlı parametre dizisi olarak ele alınır:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //valid
```

Aşırı yüklemelere sahip işlevler, aşırı yükleme imzası uygulama imzasıyla uyumluysa geçerlidir:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Valid
x('a', 1); // Valid

function y(a: string): void; // Invalid, not compatible with
implementation signature
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
```

```
y('a');
y('a', 1);
```

Kaynak ve hedef parametreler üst türlere veya alt türlere atanabiliyorsa işlev parametresi karşılaştırması başarılı olur (çift değişkenlik).

```
// Supertype
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Subtype
class Y extends X {}
// Subtype
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// Bivariance does accept supertypes
console.log(getA(new X('x'))); // Valid
console.log(getA(new Y('Y'))); // Valid
console.log(getA(new Z('z'))); // Valid
```

Enum'lar sayılarla, sayılar da Enum'larla karşılaştırılabilir ve geçerlidir; ancak farklı Enum türlerinden Enum değerlerini karşılaştırmak geçersizdir.

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
```

```
    C,
}
const xa: number = X.A; // Valid
const ya: Y = 0; // Valid
X.A === Y.A; // Invalid
```

Bir sınıfın örnekleri, private ve protected üyeleri için uyumluluk denetimine tabi tutulur:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // Invalid
```

Karşılaştırma denetimi farklı kalıtım hiyerarşilerini dikkate almaz, örneğin:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
```

```
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Valid
x === z; // Valid even if z is from a different inheritance hierarchy
```

Jenerikler, jenerik parametre uygulandıktan sonra ortaya çıkan türün yapısı kullanılarak karşılaştırılır; yalnızca nihai sonuç jenerik olmayan bir tür olarak karşılaştırılır.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Invalid as the type argument is used in the final structure

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Valid as the type argument is not used in the final structure
```

Jeneriklerin tür argümanı belirtilmediğinde, belirtilmemiş tüm argümanlar “any” türü olarak ele alınır:

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
```

```
let x: X = x => x;
let y: Y = y => y;
x = y; // Valid
```

Unutmayın:

```
let a: number = 1;
let b: number = 2;
a = b; // Valid, everything is assignable to itself

let c: any;
c = 1; // Valid, all types are assignable to any

let d: unknown;
d = 1; // Valid, all types are assignable to unknown

let e: unknown;
let e1: unknown = e; // Valid, unknown is only assignable to
itself and any
let e2: any = e; // Valid
let e3: number = e; // Invalid

let f: never;
f = 1; // Invalid, nothing is assignable to never

let g: void;
let g1: any;
g = 1; // Invalid, void is not assignable to or from anything
except any
g = g1; // Valid
```

“strictNullChecks” etkinleştirildiğinde “null” ve “undefined” değerlerinin “void” değerine benzer şekilde, aksi durumda ise “never” değerine benzer şekilde ele alındığını lütfen unutmayın.

Kümeler Olarak Türler

TypeScript'te bir tür, olası değerlerin oluşturduğu bir kümedir. Bu küme, türün etki alanı olarak da adlandırılır. Bir türün her değeri, bir kümenin elemanı olarak görülebilir. Bir tür, kümedeki her elemanın o kümenin bir üyesi sayılması için karşılaması gereken kısıtlamaları belirler. TypeScript'in temel görevi, bir kümenin diğerinin alt kümesi olup olmadığını denetlemek ve doğrulamaktır.

TypeScript çeşitli küme türlerini destekler:

Küme terimi	**TypeScript**	**Notlar**
Boş küme	never	"never", kendisi dışında hiçbir şey içermez
Tek elemanlı küme	undefined / null / literal type	
Sonlu küme	boolean / union	
Sonsuz küme	string / number / object	
Evrensel küme	any / unknown	Her eleman "any" türünün üyesidir ve her küme onun alt kümesidir / "unknown", "any" türünün tür açısından güvenli karşılığıdır

İşte birkaç örnek:

TypeScript	**Küme terimi**	**Örnek**
never	∅ (boş küme)	const x: never = 'x'; // Error: Type 'string' is not assignable to type 'never'
Değişmez tür	Tek elemanlı küme	type X = 'X'; type Y = 7;

T'ye atanabilir değer	Değer ∈ T (üyesi)	type XY = 'X' | 'Y'; const x: XY = 'X';
T1, T2'ye atanabilir	T1 ⊆ T2 (alt kümesi)	type XY = 'X' | 'Y'; const x: XY = 'X'; const j: XY = 'J'; // Type '"J"' is not assignable to type 'XY'.
T1 extends T2	T1 ⊆ T2 (alt kümesi)	type X = 'X' extends string ? true : false;
T1 | T2	T1 ∪ T2 (birleşim)	type XY = 'X' | 'Y'; type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (kesişim)	type X = { a: string } type Y = { b: string } type XY = X & Y const x: XY = { a: 'a', b: 'b' }
unknown	Evrensel küme	const x: unknown = 1

Bir birleşim, (T1 | T2), daha geniş bir küme (her ikisi) oluşturur:

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Valid
```

Bir kesişim, (T1 & T2), daha dar bir küme (yalnızca ortak olanlar) oluşturur:

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Invalid
const j: XY = { a: 'a', b: 'b' }; // Valid
```

`extends` anahtar sözcüğü bu bağlamda "alt kümesi" olarak düşünülebilir. Bir tür için kısıtlama belirler. `extends` bir jenerikle kullanıldığında, jenerik tür parametresini daha belirli bir türle sınırlar.

Buradaki `extends` ifadesinin OOP anlamında sınıf kalıtımıyla hiçbir ilgisi olmadığını lütfen unutmayın.

TypeScript yapısal türlerle çalışır ve katı bir nominal hiyerarşiye sahip değildir. Aslında aşağıdaki örnekte olduğu gibi, TypeScript nesnelerin yapısını veya biçimini dikkate aldığından iki türden biri diğerinin alt türü olmadan da bu iki tür birbiriyle örtüşebilir.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
```

```
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Valid
```

Tür Atama: Tür Bildirimleri ve Tür Onaylamaları

TypeScript'te bir tür farklı yollarla atanabilir:

Tür Bildirimi

Aşağıdaki örnekte x değişkeni için bir tür bildirmek üzere x: X (": Type") kullanıyoruz.

```
type X = {
    a: string;
};

// Type declaration
const x: X = {
    a: 'a',
};
```

Değişken belirtilen biçimde değilse TypeScript bir hata bildirir. Örneğin:

```
type X = {
    a: string;
```

```
};

const x: X = {
    a: 'a',
     b: 'b', // Error: Object literal may only specify known
properties
};
```

Tür Onaylaması

`as` anahtar sözcüğünü kullanarak bir tür onaylaması eklemek mümkündür. Bu, derleyiciye geliştiricinin bir tür hakkında daha fazla bilgiye sahip olduğunu bildirir ve oluşabilecek hataları susturur.

Örneğin:

```
type X = {
    a: string;
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

Yukarıdaki örnekte x nesnesinin X türüne sahip olduğu as anahtar sözcüğü kullanılarak onaylanır. Bu, tür tanımında bulunmayan ek bir b özelliğine sahip olsa da nesnenin belirtilen türe uyduğunu TypeScript derleyicisine bildirir.

Tür onaylamaları, özellikle DOM ile çalışırken daha belirli bir türün belirtilmesi gereken durumlarda yararlıdır. Örneğin:

```
const myInput = document.getElementById('my_input') as
HTMLInputElement;
```

Burada as HTMLInputElement tür onaylaması, TypeScript'e getElementById sonucunun bir HTMLInputElement olarak ele alınması gerektiğini bildirmek için kullanılır. Tür onaylamaları, aşağıdaki şablon dizgesi örneğinde gösterildiği gibi anahtarları yeniden eşlemek için de kullanılabilir:

```
type J<Type> = {
    [Property in keyof Type as `prefix_${string &
        Property}`]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

Bu örnekte `J<Type>` türü, Type öğesinin anahtarlarını yeniden eşlemek için şablon dizgesiyle birlikte eşlenmiş bir tür kullanır. Her anahtara "prefix_" ekleyerek yeni özellikler oluşturur ve bunlara karşılık gelen değerler, özgün özellik değerlerini döndüren işlevlerdir.

Bir tür onaylaması kullanıldığında TypeScript'in fazla özellik denetimi yapmayacağını belirtmek gerekir. Bu nedenle nesnenin yapısı önceden biliniyorsa genellikle bir Tür Bildirimi kullanılması tercih edilir.

Ortam Bildirimleri

Ortam bildirimleri, JavaScript kodunun türlerini tanımlayan dosyalardır ve dosya adı biçimleri `.d.ts.` şeklindedir. Genellikle mevcut JavaScript kütüphanelerine açıklama eklemek veya projenizdeki mevcut JS dosyalarına tür eklemek için içe aktarılır ve kullanılır.

Birçok yaygın kütüphanenin türleri şu adreste bulunabilir: https://github.com/DefinitelyTyped/DefinitelyTyped/

ve şu komutla yüklenebilir:

```
npm install --save-dev @types/library-name
```

Kendi tanımladığınız Ortam Bildirimlerini "üç eğik çizgili" başvuruyu kullanarak içe aktarabilirsiniz:

```
/// <reference path="./library-types.d.ts" />
```

Ortam Bildirimlerini `// @ts-check` kullanarak JavaScript dosyalarının içinde bile kullanabilirsiniz.

`declare` anahtar sözcüğü, mevcut JavaScript kodunu içe aktarmadan onun için tür tanımları yapılmasını sağlar ve başka bir dosyadaki ya da genel kapsamdaki türler için yer tutucu görevi görür.

Özellik Denetimi ve Fazla Özellik Denetimi

TypeScript yapısal bir tür sistemini temel alır; ancak fazla özellik denetimi, bir nesnenin türde belirtilen özelliklere tam olarak sahip olup olmadığını denetlemesini sağlayan bir TypeScript özelliğidir.

Fazla Özellik Denetimi, nesne değişmezleri değişkenlere atanırken veya işlevin fazla özelliğine argüman olarak aktarılırken gerçekleştirilir. Örneğin:

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
```

```
const x: X = y; // Valid because structural typing
const w: X = { a: 'a', b: 'b' }; // Invalid because excess
property checking
```

Zayıf Türler

Bir tür, yalnızca tümü isteğe bağlı olan bir özellik kümesi içerdiğinde zayıf kabul edilir:

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript, hiçbir örtüşme olmadığında zayıf bir türe herhangi bir şey atanmasını hata olarak değerlendirir; örneğin aşağıdaki kod hata verir:

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Invalid
```

Önerilmemekle birlikte, gerekirse tür onaylaması kullanarak bu denetimi atlamak mümkündür:

```
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Valid
```

Ya da zayıf türün indeks imzasına unknown ekleyerek:

```
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Valid
```

Katı Nesne Değişmezi Denetimi (Tazelik)

Bazen "tazelik" olarak adlandırılan katı nesne değişmezi denetimi, normal yapısal tür denetimlerinde aksi hâlde fark edilmeyecek fazla veya yanlış yazılmış özellikleri yakalamaya yardımcı olan bir TypeScript özelliğidir.

Bir nesne değişmezi oluşturulurken TypeScript derleyicisi onu "taze" kabul eder. Nesne değişmezi bir değişkene atanır veya parametre olarak aktarılırsa ve hedef türde bulunmayan özellikler belirtirse TypeScript bir hata verir.

Ancak bir nesne değişmezi genişletildiğinde veya bir tür onaylaması kullanıldığında "tazelik" ortadan kalkar.

İşte bunu gösteren bazı örnekler:

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Freshness check: Invalid assignment
var y: Y;
y = { a: 'a', bx: 'bx' }; // Freshness check: Invalid
assignment
```

```
const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Widening: No errors, structurally type compatible

fn({ a: 'a', bx: 'b' }); // Freshness check: Invalid argument

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Widening: No Freshness check
```

Tür Çıkarımı

TypeScript, şu durumlarda ek açıklama sağlanmadığında türleri çıkarabilir:

- Değişken başlatma.
- Üye başlatma.
- Parametreler için varsayılan değerleri ayarlama.
- İşlev dönüş türü.

Örneğin:

```
let x = 'x'; // The type inferred is string
```

TypeScript derleyicisi, değeri veya ifadeyi analiz eder ve mevcut bilgilere göre türünü belirler.

Daha Gelişmiş Çıkarımlar

Tür çıkarımında birden fazla ifade kullanıldığında TypeScript “en iyi ortak türleri” arar. Örneğin:

```
let x = [1, 'x', 1, null]; // The type inferred is: (string | number | null)[]
```

Derleyici en iyi ortak türleri bulamazsa bir birleşim türü döndürür. Örneğin:

```
let x = [new RegExp('x'), new Date()]; // Type inferred is: (RegExp | Date)[]
```

TypeScript, türleri çıkarmak için değişkenin konumuna dayalı "bağlamsal türleme" kullanır. Aşağıdaki örnekte derleyici, `e` öğesinin `MouseEvent` türünde olduğunu bilir; çünkü çeşitli yaygın JavaScript yapılarına ve DOM'a yönelik ortam bildirimlerini içeren lib.d.ts dosyasında `click` olay türü tanımlanmıştır:

```
window.addEventListener('click', function (e) {}); // The inferred type of e is MouseEvent
```

Tür Genişletme

Tür genişletme, TypeScript'in tür ek açıklaması olmadan başlatılan bir değişkene tür atadığı süreçtir. Dar türlerden daha geniş türlere geçişe izin verir, ancak tersine izin vermez. Aşağıdaki örnekte:

```
let x = 'x'; // TypeScript infers as string, a wide type
let y: 'y' | 'x' = 'y'; // y types is a union of literal types
y = x; // Invalid Type 'string' is not assignable to type '"x" | "y"'.
```

TypeScript, `string` türünü `x` değişkenine, başlatma sırasında sağlanan tek değeri (`x`) temel alarak atar; bu bir genişletme örneğidir.

TypeScript, örneğin "const" kullanarak genişletme sürecini denetlemenin yollarını sağlar.

Const

Bir değişken bildirilirken `const` anahtar sözcüğünün kullanılması, TypeScript'te daha dar bir tür çıkarımıyla sonuçlanır.

Örneğin:

```
const x = 'x'; // TypeScript infers the type of x as 'x', a narrower type
let y: 'y' | 'x' = 'y';
y = x; // Valid: The type of x is inferred as 'x'
```

x değişkenini bildirmek için `const` kullanıldığında, türü belirli 'x' değişmez değerine daraltılır. x türü daraltıldığından, herhangi bir hata olmadan y değişkenine atanabilir. Türün çıkarılabilmesinin nedeni, `const` değişkenlerinin yeniden atanamaması ve dolayısıyla türlerinin belirli bir değişmez türe, bu durumda 'x' değişmez türüne, daraltılabilmesidir.

Tür Parametrelerinde Const Değiştiricisi

TypeScript'in 5.0 sürümünden itibaren bir jenerik tür parametresinde `const` niteliğini belirtmek mümkündür. Bu, mümkün olan en kesin türün çıkarılmasını sağlar. `const` kullanmadan bir örnek görelim:

```
function identity<T>(value: T) {
    // No const here
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred is: { a: string; b: string; }
```

Gördüğünüz gibi `a` ve `b` özelliklerinin türü `string` olarak çıkarılır.

Şimdi `const` sürümüyle arasındaki farkı görelim:

```
function identity<const T>(value: T) {
    // Using const modifier on type parameters
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred
is: { a: "a"; b: "b"; }
```

Artık `a` ve `b` özelliklerinin yalnızca `string` türleri yerine dize değişmezleri olarak çıkarıldığını görebiliriz.

Const Onaylaması

Bu özellik, başlatma değerine dayalı daha kesin bir değişmez türle değişken bildirmenize olanak tanır ve derleyiciye değerin değiştirilemez bir değişmez değer olarak ele alınması gerektiğini belirtir. İşte birkaç örnek:

Tek bir özellikte:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

Bir nesnenin tamamında:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

Bu, özellikle bir tuple için tür tanımlarken yararlı olabilir:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tuple of readonly [1, 2, 3]
```

Açık Tür Ek Açıklaması

Daha belirli davranıp bir tür aktarabiliriz. Aşağıdaki örnekte `x` özelliği `number` türündedir:

```
const v = {
    x: 1, // Inferred type: number (widening)
};
v.x = 3; // Valid
```

Değişmez türlerin birleşimini kullanarak tür ek açıklamasını daha belirli hâle getirebiliriz:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x is now a union of literal types: 1 | 2 | 3
};
v.x = 3; // Valid
v.x = 100; // Invalid
```

Tür Daraltma

Tür Daraltma, TypeScript'te genel bir türün daha belirli bir türe daraltıldığı süreçtir. Bu, TypeScript kodu analiz edip belirli koşulların veya işlemlerin tür bilgilerini iyileştirebileceğini belirlediğinde gerçekleşir.

Türleri daraltma farklı şekillerde gerçekleşebilir; bunlar arasında şunlar yer alır:

Koşullar

TypeScript, `if` veya `switch` gibi koşullu ifadeler kullanıldığında koşulun sonucuna göre türü daraltabilir. Örneğin:

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // The type is number, which had been narrowed by
the condition
}
```

Hata fırlatma veya dönüş

Bir hata fırlatmak veya bir daldan erken dönmek, TypeScript'in bir türü daraltmasına yardımcı olmak için kullanılabilir. Örneğin:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

TypeScript'te türleri daraltmanın diğer yolları şunlardır:

- `instanceof` operatörü: Bir nesnenin belirli bir sınıfın örneği olup olmadığını denetlemek için kullanılır.
- `in` operatörü: Bir nesnede bir özelliğin bulunup bulunmadığını denetlemek için kullanılır.
- `typeof` operatörü: Çalışma zamanında bir değerin türünü kontrol etmek için kullanılır.
- `Array.isArray()` gibi yerleşik fonksiyonlar: Bir değerin dizi olup olmadığını kontrol etmek için kullanılır.

Ayırt Edici Birleşim

"Ayırt Edici Birleşim" kullanmak, TypeScript'te bir birleşim içindeki farklı türleri birbirinden ayırmak için nesnelere açık

bir “etiket” eklendiği bir kalıptır. Bu kalıp “etiketli birleşim” olarak da adlandırılır. Aşağıdaki örnekte “etiket”, “type” özelliğiyle temsil edilir:

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // type is A
        case 'type_b':
            return input.value + 'extra'; // type is B
    }
};
```

Kullanıcı Tanımlı Tür Korumaları

TypeScript’in bir türü belirleyemediği durumlarda, “kullanıcı tanımlı tür koruması” olarak bilinen bir yardımcı fonksiyon yazmak mümkündür. Aşağıdaki örnekte, belirli bir filtreleme uyguladıktan sonra türü daraltmak için bir Tür Yükleminden yararlanacağız:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // The type is (string
| null)[], TypeScript was not able to infer the type properly

const isValid = (item: string | null): item is string => item
!== null; // Custom type guard

const r2 = data.filter(isValid); // The type is fine now
string[], by using the predicate type guard we were able to
narrow the type
```

switch-true daraltması

TypeScript 5.3, karmaşık if/else zincirlerini boole koşullarıyla switch (true) kullanarak değiştirmenize olanak tanıyan switch-true daraltmasını ekler. Okunabilirliği artırır ve yine de türleri daraltır. Örüntü eşlemeye benzer, ancak daha basittir.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

İlkel Türler

TypeScript 7 ilkel türü destekler. İlkel veri türü, nesne olmayan ve kendisiyle ilişkili herhangi bir yöntemi bulunmayan bir türü ifade eder. TypeScript'teki tüm ilkel türler değişmezdir; yani değerleri atandıktan sonra değiştirilemez.

string

`string` ilkel türü metinsel verileri saklar ve değer her zaman çift ya da tek tırnak içine alınır.

```
const x: string = 'x';
const y: string = 'y';
```

Dizeler, ters tırnak (`) karakteriyle çevrelenirse birden fazla satıra yayılabilir:

```
let sentence: string = `xxx,
   yyy`;
```

boolean

TypeScript'teki `boolean` veri türü, `true` veya `false` olmak üzere ikili bir değer saklar.

```
const isReady: boolean = true;
```

number

TypeScript'teki `number` veri türü, 64 bitlik kayan noktalı bir değerle temsil edilir. `number` türü tam sayıları ve kesirleri temsil edebilir. TypeScript ayrıca onaltılık, ikilik ve sekizlik sayıları da destekler; örneğin:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Hexadecimal starts with
0x
const binary: number = 0b1010; // Binary starts with 0b
const octal: number = 0o633; // Octal starts with 0o
```

bigint

`bigint`, `number` tarafından desteklenen ve 2^53 - 1 olan en büyük güvenli tam sayıdan daha büyük tam sayı değerlerini temsil eder.

Bir `bigint`, yerleşik `BigInt()` fonksiyonu çağrılarak veya herhangi bir tam sayı sayısal sabitinin sonuna `n` eklenerek oluşturulabilir:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Notlar:

- `bigint` değerleri `number` ile karıştırılamaz ve yerleşik `Math` ile kullanılamaz; aynı türe dönüştürülmeleri gerekir.
- `bigint` değerleri yalnızca hedef yapılandırması ES2020 veya üzeriyse kullanılabilir.

Symbol

Semboller, adlandırma çakışmalarını önlemek için nesnelerde özellik anahtarı olarak kullanılabilen benzersiz tanımlayıcılardır.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null ve undefined

Hem `null` hem de `undefined` türleri, bir değerin bulunmamasını veya herhangi bir değerin yokluğunu temsil eder.

`undefined` türü, değerin atanmadığı ya da başlatılmadığı anlamına gelir veya bir değerin kasıtsız olarak bulunmadığını

belirtir.

`null` türü, alanın bir değeri olmadığını bildiğimiz için değerin kullanılamadığını ifade eder ve bir değerin kasıtlı olarak bulunmadığını belirtir.

Dizi

Bir `array`, aynı türde veya farklı türlerde birden fazla değeri saklayabilen bir veri türüdür. Aşağıdaki söz dizimi kullanılarak tanımlanabilir:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

TypeScript, aşağıdaki söz dizimini kullanarak salt okunur dizileri destekler:

```
const x: readonly string[] = ['a', 'b']; // Readonly modifier
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Invalid
```

TypeScript, demetleri ve salt okunur demetleri destekler:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

`any` veri türü, kelimenin tam anlamıyla “herhangi bir” değeri temsil eder ve TypeScript türü çıkaramadığında ya da tür belirtilmediğinde varsayılandır.

`any` kullanıldığında TypeScript derleyicisi tür denetimini atlar; dolayısıyla `any` kullanımı sırasında tür güvenliği yoktur. Genel olarak, bir hata oluştuğunda derleyiciyi susturmak için `any` kullanmayın; bunun yerine hatayı düzeltmeye odaklanın. Çünkü `any` kullanmak sözleşmelerin bozulmasına ve TypeScript otomatik tamamlamasının avantajlarının kaybedilmesine yol açabilir.

`any` türü, derleyiciyi susturabildiği için JavaScript'ten TypeScript'e kademeli bir geçiş sırasında faydalı olabilir.

Yeni projelerde `noImplicitAny` TypeScript yapılandırmasını kullanın; bu yapılandırma, `any` kullanıldığı veya çıkarıldığı yerlerde TypeScript'in hata vermesini sağlar.

`any` türü genellikle türlerinizdeki gerçek sorunları maskeleyebilen bir hata kaynağıdır. Mümkün olduğunca kullanmaktan kaçının.

Tür Ek Açıklamaları

`var`, `let` ve `const` kullanılarak bildirilen değişkenlere isteğe bağlı olarak bir tür eklemek mümkündür:

```
const x: number = 1;
```

TypeScript, özellikle basit türler için tür çıkarımını iyi yaptığından bu bildirimler çoğu durumda gerekli değildir.

Fonksiyonlarda parametrelere tür ek açıklamaları eklemek mümkündür:

```
function sum(a: number, b: number) {
    return a + b;
```

```
}
```

Aşağıda anonim bir fonksiyonun (lambda fonksiyonu olarak da adlandırılır) kullanıldığı bir örnek yer almaktadır:

```
const sum = (a: number, b: number) => a + b;
```

Bir parametre için varsayılan değer bulunduğunda bu ek açıklamalardan kaçınılabilir:

```
const sum = (a = 10, b: number) => a + b;
```

Fonksiyonlara dönüş türü ek açıklamaları eklenebilir:

```
const sum = (a = 10, b: number): number => a + b;
```

Bu, özellikle daha karmaşık fonksiyonlar için faydalıdır; çünkü uygulamadan önce dönüş türünü yazmak fonksiyon üzerinde düşünmenize yardımcı olabilir.

Genel olarak, tür imzalarına ek açıklama eklemeyi değerlendirin, ancak fonksiyon gövdesindeki yerel değişkenlere eklemeyin ve nesne sabitlerine her zaman tür ekleyin.

İsteğe Bağlı Özellikler

Bir nesne, özellik adının sonuna soru işareti `?` ekleyerek İsteğe Bağlı Özellikler belirtebilir:

```
type X = {
    a: number;
    b?: number; // Optional
};
```

Bir özellik isteğe bağlı olduğunda varsayılan değer belirtmek mümkündür:

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

Salt Okunur Özellikler

Bir özelliğin yeniden yazılamamasını sağlayan ancak tamamen değişmez olduğuna dair herhangi bir güvence vermeyen `readonly` değiştiricisini kullanarak özelliğe yazmayı önlemek mümkündür:

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

Dizin İmzaları

TypeScript'te dizin imzası olarak `string`, `number` ve `symbol` kullanabiliriz:

```
type K = {
    [name: string | number]: string;
```

```
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Same result as k[1]
```

JavaScript'in `number` türündeki bir dizini otomatik olarak `string` türündeki bir dizine dönüştürdüğünü lütfen unutmayın; bu nedenle `k[1]` veya `k["1"]` aynı değeri döndürür.

Türleri Genişletme

Bir `interface`'i genişletmek (üyeleri başka bir türden kopyalamak) mümkündür:

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

Birden fazla türü genişletmek de mümkündür:

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

`extends` anahtar sözcüğü yalnızca arayüzlerde ve sınıflarda çalışır; türler için kesişim kullanın:

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

Bir türü arayüz kullanarak genişletmek mümkündür, ancak bunun tersi mümkün değildir:

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

Sabit Değer Türleri

Sabit Değer Türü, toplu bir tür içindeki tek öğeli bir kümedir; JavaScript ilkel türü olan son derece kesin bir değeri tanımlar.

TypeScript'teki Sabit Değer Türleri sayılar, dizeler ve boole değerleridir.

Sabit değer örnekleri:

```
const a = 'a'; // String literal type
const b = 1; // Numeric literal type
const c = true; // Boolean literal type
```

Dize, Sayısal ve Boole Sabit Değer Türleri birleşimlerde, tür korumalarında ve tür takma adlarında kullanılır. Aşağıdaki örnekte bir birleşim türü takma adı görebilirsiniz. `o` yalnızca belirtilen değerlerden oluşur; başka hiçbir dize geçerli değildir:

```
type O = 'a' | 'b' | 'c';
```

Sabit Değer Çıkarımı

Sabit Değer Çıkarımı, bir değişkenin veya parametrenin türünün değerine göre çıkarılmasına olanak tanıyan bir TypeScript özelliğidir.

Aşağıdaki örnekte TypeScript'in, değeri daha sonra değiştirilemeyeceği için `x`'i sabit değer türü olarak değerlendirdiğini; buna karşılık `y` daha sonra herhangi bir zamanda değiştirilebildiği için türünün string olarak çıkarıldığını görebiliriz.

```
const x = 'x'; // Literal type of 'x', because this value cannot be changed
let y = 'y'; // Type string, as we can change this value
```

Aşağıdaki örnekte, TypeScript değerin daha sonra herhangi bir zamanda değiştirilebileceğini düşündüğünden `o.x` türünün `string` olarak (`a` sabit değeri olarak değil) çıkarıldığını görebiliriz.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // This is a wider string
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // Argument of type 'string' is not assignable to parameter of type 'X'
```

Gördüğünüz gibi, X daha dar bir tür olduğundan `o.x`, `fn`'ye geçirildiğinde kod hata verir.

Bu sorunu `const` veya `X` türüyle bir tür onaylaması kullanarak çözebiliriz:

```
let o = {
    x: 'a' as const,
};
```

veya:

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

`strictNullChecks`, katı null denetimini zorunlu kılan bir TypeScript derleyici seçeneğidir. Bu seçenek etkinleştirildiğinde, değişkenlere ve parametrelere yalnızca `null | undefined` birleşim türü kullanılarak açıkça bu türde bildirildikleri takdirde `null` veya `undefined` atanabilir. Bir değişken ya da parametre açıkça null atanabilir olarak bildirilmemişse TypeScript, olası çalışma zamanı hatalarını önlemek için bir hata oluşturur.

Enum'lar

TypeScript'te bir `enum`, adlandırılmış sabit değerler kümesidir.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
```

```
    Blue = '#0000ff',
}
```

Enum'lar farklı şekillerde tanımlanabilir:

Sayısal enum'lar

TypeScript'te Sayısal Enum, her sabite varsayılan olarak 0'dan başlayan sayısal bir değerin atandığı bir Enum'dur.

```
enum Size {
    Small, // value starts from 0
    Medium,
    Large,
}
```

Değerleri açıkça atayarak özel değerler belirtmek mümkündür:

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

Dize enum'ları

TypeScript'te Dize Enum'u, her sabite bir dize değerinin atandığı bir Enum'dur.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Not: TypeScript, dize ve sayısal üyelerin bir arada bulunabildiği heterojen Enum'ların kullanımına izin verir.

Sabit enum'lar

TypeScript'te sabit enum, tüm değerlerin derleme zamanında bilindiği ve enum'un kullanıldığı her yerde satır içine alındığı, böylece daha verimli kod elde edilen özel bir Enum türüdür.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

Şuna derlenir:

```
console.log('EN' /* Language.English */);
```

Notlar: Const Enum'lar sabit kodlanmış değerlere sahiptir ve Enum'u siler; bu, kendi içinde bağımsız kitaplıklarda daha verimli olabilir, ancak genellikle tercih edilmez. Ayrıca Const enum'lar hesaplanmış üyelere sahip olamaz.

Ters eşleme

TypeScript'te Enum'lardaki ters eşlemeler, Enum üyesinin adını değerinden alma yeteneğini ifade eder. Varsayılan olarak Enum üyelerinde addan değere ileri eşlemeler bulunur, ancak her üye için değerler açıkça ayarlanarak ters eşlemeler oluşturulabilir. Ters eşlemeler, bir Enum üyesini değerine göre aramanız veya tüm Enum üyeleri üzerinde yineleme yapmanız gerektiğinde kullanışlıdır. Yalnızca sayısal enum üyelerinin ters eşlemeler oluşturacağını, dize enum üyeleri içinse hiçbir ters eşleme oluşturulmayacağını unutmayın.

Aşağıdaki enum:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

Şuna derlenir:

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Dolayısıyla değerlerin anahtarlarla eşlenmesi sayısal enum üyeleri için çalışırken dize enum üyeleri için çalışmaz:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]);  //  Element  implicitly  has  an
'any' type because index expression is not of type 'number'.
```

Bildirimsel enum'lar

TypeScript'te bildirimsel enum, ilişkili bir uygulaması olmadan bir bildirim dosyasında (*.d.ts) tanımlanan bir Enum türüdür. Her dosyada uygulama ayrıntılarını içe aktarmak zorunda kalmadan farklı dosyalar arasında tür açısından güvenli biçimde kullanılabilen adlandırılmış sabitler kümesi tanımlamanıza olanak tanır.

Hesaplanmış ve sabit üyeler

TypeScript'te hesaplanmış üye, değeri çalışma zamanında hesaplanan bir Enum üyesiyken sabit üye, değeri derleme zamanında belirlenen ve çalışma zamanında değiştirilemeyen bir üyedir. Hesaplanmış üyelere normal Enum'larda izin verilirken sabit üyelere hem normal hem de const enum'larda izin verilir.

```
// Constant members
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 generation at compilation time

// Computed members
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // random number generated at run time
```

Enum'lar, üye türlerinden oluşan birleşimlerle gösterilir. Her üyenin değeri sabit veya sabit olmayan ifadeler aracılığıyla belirlenebilir ve sabit değerlere sahip üyelere sabit değer

türleri atanır. Örnek olarak E türünün ve E.A, E.B ve E.C alt türlerinin bildirimini ele alalım. Bu durumda E, E.A | E.B | E.C birleşimini temsil eder.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Numeric literal
    B = 'bar', // String literal
    C = identity(42), // Opaque computed
}

console.log(E.C); //42
```

Daraltma

TypeScript daraltması, koşullu bir blok içinde bir değişkenin türünü daha belirgin hâle getirme işlemidir. Bu, bir değişkenin birden fazla türe sahip olabildiği birleşim türleriyle çalışırken kullanışlıdır.

TypeScript, türü daraltmanın çeşitli yollarını tanır:

typeof tür korumaları

typeof tür koruması, TypeScript'te bir değişkenin türünü yerleşik JavaScript türüne göre kontrol eden belirli bir tür korumasıdır.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x is number
    }
    return -1;
};
```

Doğruluk değeriyle daraltma

TypeScript'te doğruluk değeriyle daraltma, bir değişkenin türünü buna göre daraltmak için değişkenin doğru veya yanlış olarak değerlendirilip değerlendirilmediğini kontrol ederek çalışır.

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

Eşitlikle daraltma

TypeScript'te eşitlikle daraltma, bir değişkenin belirli bir değere eşit olup olmadığını kontrol ederek türünü buna göre daraltır.

Türleri daraltmak için `switch` ifadeleriyle ve `===`, `!==`, `==` ve `!=` gibi eşitlik operatörleriyle birlikte kullanılır.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

in Operatörüyle daraltma

TypeScript'te `in` Operatörüyle daraltma, değişkenin türünde bir özelliğin bulunup bulunmadığına göre değişkenin türünü

daraltmanın bir yoludur.

```
type Dog = {
    name: string;
    breed: string;
};

type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

instanceof ile daraltma

TypeScript'te `instanceof` operatörüyle daraltma, bir nesnenin belirli bir sınıfın veya arayüzün örneği olup olmadığını kontrol ederek, değişkenin türünü oluşturucu fonksiyonuna göre daraltmanın bir yoludur.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
```

```
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Atamalar

Atamaları kullanan TypeScript daraltması, bir değişkenin türünü kendisine atanan değere göre daraltmanın bir yoludur. Bir değişkene bir değer atandığında TypeScript, atanan değere göre türünü çıkarır ve değişkenin türünü çıkarılan türle eşleşecek biçimde daraltır.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Kontrol Akışı Analizi

TypeScript'te Kontrol Akışı Analizi, değişkenlerin türlerini çıkarmak için kod akışını statik olarak analiz etmenin ve analiz sonuçlarına göre derleyicinin bu değişkenlerin türlerini gerektiği gibi daraltmasına olanak tanımanın bir yoludur.

TypeScript 4.4'ten önce kod akışı analizi yalnızca bir if ifadesi içindeki koda uygulanıyordu; ancak TypeScript 4.4'ten itibaren const değişkenleri üzerinden dolaylı olarak başvurulan koşullu ifadelere ve ayırt edici özellik erişimlerine de uygulanabilir.

Örneğin:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Daraltmanın gerçekleşmediği bazı örnekler:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
        x.length; // Error, no narrowing because isString it is
not const
    }
};

const f6 = (
```

```
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
        obj.foo; // Error, no narrowing because obj is assigned
in function body
    }
};
```

Notlar: Koşullu ifadelerde en fazla beş dolaylılık düzeyi analiz edilir.

Tür Yüklemleri

TypeScript'te Tür Yüklemleri, boole değeri döndüren ve bir değişkenin türünü daha belirli bir türe daraltmak için kullanılan fonksiyonlardır.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5, `x is T` gibi tür yüklemlerini `.filter` gibi fonksiyonlarda otomatik olarak çıkarır. Böylece undefined gibi değerlerin ne zaman kaldırıldığını bilir; bu da daha kesin türler ve daha az hata sağlar. Bu özellik açık denetimlerde (ör. `x !== undefined`) çalışır, ancak `!!x` gibi belirsiz denetimlerde çalışmaz.

```
const nums = [1, null, 2].filter(x => x !== null);
```

Ayırt Edici Birleşimler

TypeScript'te Ayırt Edici Birleşimler, birleşim için olası türler kümesini daraltmak üzere ayırt edici olarak bilinen ortak bir özellik kullanan birleşim türleridir.

```
type Square = {
    kind: 'square'; // Discriminant
    size: number;
};

type Circle = {
    kind: 'circle'; // Discriminant
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

never Türü

Bir değişken, hiçbir değer içeremeyen bir türe daraltıldığında TypeScript derleyicisi değişkenin `never` türünde olması gerektiğini çıkarır. Bunun nedeni, never türünün hiçbir zaman üretilemeyecek bir değeri temsil etmesidir.

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
          // val has type never here because it can never be
anything other than a string or a number
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

Kapsamlılık denetimi

Kapsamlılık denetimi, ayırt edici bir birleşimin tüm olası durumlarının bir `switch` veya `if` ifadesinde ele alınmasını sağlayan bir TypeScript özelliğidir.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
```

```
                console.log(exhaustiveCheck); // This line will
never be executed
    }
};
```

`never` türü, varsayılan durumun kapsamlı olmasını ve switch ifadesinde ele alınmadan Direction türüne yeni bir değer eklenirse TypeScript'in hata vermesini sağlamak için kullanılır.

Nesne Türleri

TypeScript'te nesne türleri bir nesnenin şeklini açıklar. Nesnenin özelliklerinin adlarını ve türlerini, ayrıca bu özelliklerin zorunlu mu yoksa isteğe bağlı mı olduğunu belirtirler.

TypeScript'te nesne türlerini iki temel yolla tanımlayabilirsiniz:

Bir arayüz, özelliklerinin adlarını, türlerini ve isteğe bağlılık durumunu belirterek bir nesnenin şeklini tanımlar.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

Bir arayüze benzer şekilde tür takma adı da bir nesnenin şeklini tanımlar. Ancak mevcut bir türe veya mevcut türlerin birleşimine dayanan yeni ve özel bir tür de oluşturabilir. Buna birleşim türlerinin, kesişim türlerinin ve diğer karmaşık türlerin tanımlanması dahildir.

```
type Point = {
    x: number;
```

```
    y: number;
};
```

Bir türü anonim olarak tanımlamak da mümkündür:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

Demet Türü (Anonim)

Demet Türü, sabit sayıda öğe ve bunlara karşılık gelen türleri içeren bir diziyi temsil eden türdür. Demet türü, sabit bir sırada belirli sayıda öğeyi ve bunların ilgili türlerini zorunlu kılar. Demet türleri, her öğenin dizideki konumunun belirli bir anlama sahip olduğu, belirli türlerdeki bir değer koleksiyonunu temsil etmek istediğinizde kullanışlıdır.

```
type Point = [number, number];
```

Adlandırılmış Demet Türü (Etiketli)

Demet türleri, her öğe için isteğe bağlı etiketler veya adlar içerebilir. Bu etiketler okunabilirlik ve araç desteği içindir ve bunlarla gerçekleştirebileceğiniz işlemleri etkilemez.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Named Tuple plus Anonymous Tuple
```

Sabit Uzunluklu Demet

Sabit Uzunluklu Demet, belirli türlerde sabit sayıda öğeyi zorunlu kılan ve tanımlandıktan sonra demetin uzunluğunda

herhangi bir değişikliğe izin vermeyen özel bir demet türüdür.

Sabit Uzunluklu Demetler, belirli sayıda öğe ve belirli türlerden oluşan bir değer koleksiyonunu temsil etmeniz ve demetin uzunluğu ile türlerinin yanlışlıkla değiştirilemeyeceğinden emin olmanız gerektiğinde kullanışlıdır.

```
const x = [10, 'hello'] as const;
x.push(2); // Error
```

Birleşim Türü

Birleşim Türü, birkaç türden biri olabilen bir değeri temsil eden türdür. Birleşim Türleri, olası her türün arasında | simgesi kullanılarak gösterilir.

```
let x: string | number;
x = 'hello'; // Valid
x = 123; // Valid
```

Kesişim Türleri

Kesişim Türü, iki veya daha fazla türün tüm özelliklerine sahip bir değeri temsil eden türdür. Kesişim Türleri, her türün arasında & simgesi kullanılarak gösterilir.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Intersection
```

```
const j: J = {
    a: 'a',
    b: 'b',
};
```

Tür Dizinleme

Tür dizinleme, açıkça bildirilmemiş özelliklerin türünü belirtmek için bir dizin imzası kullanarak önceden bilinmeyen bir anahtarla dizinlenebilen türleri tanımlama yeteneğini ifade eder.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Returns a
```

Değerden Tür

TypeScript'te Değerden Tür, tür çıkarımı aracılığıyla bir değer veya ifadeden türün otomatik olarak çıkarılmasını ifade eder.

```
const x = 'x'; // TypeScript infers 'x' as a string literal with 'const' (immutable), but widens it to 'string' with 'let' (reassignable).
```

Fonksiyon Dönüşünden Tür

Fonksiyon Dönüşünden Tür, bir fonksiyonun dönüş türünü uygulamasına göre otomatik olarak çıkarma yeteneğini ifade eder. Bu, TypeScript'in fonksiyon tarafından döndürülen değerin türünü açık tür ek açıklamaları olmadan belirlemesine olanak tanır.

```
const add = (x: number, y: number) => x + y; // TypeScript can
infer that the return type of the function is a number
```

Modülden Tür

Modülden Tür, bir modülün dışa aktarılan değerlerini kullanarak bunların türlerini otomatik olarak çıkarma yeteneğini ifade eder. Bir modül belirli bir türe sahip bir değeri dışa aktardığında TypeScript, bu değer başka bir modüle içe aktarılırken türünü otomatik olarak çıkarmak için bu bilgiyi kullanabilir.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r is number
```

Eşlenmiş Türler

TypeScript'teki Eşlenmiş Türler, her özelliği bir eşleme fonksiyonu kullanarak dönüştürüp mevcut bir türü temel alan yeni türler oluşturmanıza olanak tanır. Mevcut türleri eşleyerek aynı bilgiyi farklı bir biçimde temsil eden yeni türler oluşturabilirsiniz. Eşlenmiş bir tür oluşturmak için `keyof` operatörünü kullanarak mevcut bir türün özelliklerine erişir ve ardından yeni bir tür üretmek üzere bunları değiştirirsiniz. Aşağıdaki örnekte:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
```

```
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

MyMappedType'ı, T'nin özellikleri üzerinde eşleme yaparak her özelliğin özgün türünün bir dizisi olduğu yeni bir tür oluşturacak şekilde tanımlarız. Bunu kullanarak MyType ile aynı bilgiyi, ancak her özelliği bir dizi olacak biçimde temsil eden MyNewType'ı oluştururuz.

Eşlenmiş Tür Değiştiricileri

TypeScript'teki Eşlenmiş Tür Değiştiricileri, mevcut bir tür içindeki özelliklerin dönüştürülmesini sağlar:

- `readonly` veya `+readonly`: Bu, eşlenmiş türdeki bir özelliği salt okunur hâle getirir.
- `-readonly`: Bu, eşlenmiş türdeki bir özelliğin değiştirilebilir olmasını sağlar.
- `?`: Bu, eşlenmiş türdeki bir özelliği isteğe bağlı olarak belirler.

Örnekler:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // All properties marked as read-only

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // All properties marked as mutable

type MyPartial<T> = { [P in keyof T]?: T[P] }; // All properties marked as optional
```

Koşullu Türler

Koşullu Türler, oluşturulacak türün koşulun sonucuna göre belirlendiği, bir koşula bağlı tür oluşturmanın bir yoludur. İki tür arasında koşullu seçim yapmak için `extends` anahtar sözcüğü ve üçlü operatör kullanılarak tanımlanırlar.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // Type true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Type
false
```

Dağıtımlı Koşullu Türler

Dağıtımlı Koşullu Türler, birleşimin her üyesine ayrı ayrı bir dönüşüm uygulayarak bir türün, türler birleşimi üzerine dağıtılmasını sağlayan bir özelliktir. Bu, özellikle eşlenmiş türlerle veya yüksek dereceli türlerle çalışırken faydalı olabilir.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number |
boolean | null
```

Koşullu Türlerde infer Tür Çıkarımı

`infer` anahtar sözcüğü, koşullu türlerde bir jenerik parametrenin türünü kendisine bağlı olan bir türden çıkarmak (ayrıştırmak) için kullanılır. Bu, daha esnek ve yeniden kullanılabilir tür tanımları yazmanıza olanak tanır.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

Önceden Tanımlanmış Koşullu Türler

TypeScript'te Önceden Tanımlanmış Koşullu Türler, dil tarafından sağlanan yerleşik koşullu türlerdir. Belirli bir türün özelliklerine göre yaygın tür dönüşümlerini gerçekleştirmek üzere tasarlanmışlardır.

`Exclude<UnionType, ExcludedType>`: Bu tür, Type içindeki ExcludedType'a atanabilen tüm türleri kaldırır.

`Extract<Type, Union>`: Bu tür, Union içinden Type'a atanabilen tüm türleri çıkarır.

`NonNullable<Type>`: Bu tür, Type içinden null ve undefined değerlerini kaldırır.

`ReturnType<Type>`: Bu tür, bir Type fonksiyonunun dönüş türünü çıkarır.

`Parameters<Type>`: Bu tür, bir Type fonksiyonunun parametre türlerini çıkarır.

`Required<Type>`: Bu tür, Type içindeki tüm özellikleri zorunlu hâle getirir.

`Partial<Type>`: Bu tür, Type içindeki tüm özellikleri isteğe bağlı hâle getirir.

`Readonly<Type>`: Bu tür, Type içindeki tüm özellikleri salt okunur hâle getirir.

Şablon Birleşim Türleri

Şablon birleşim türleri, tür sistemi içinde metinleri birleştirmek ve işlemek için kullanılabilir. Örneğin:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active" | "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Any türü

`any` türü, her türlü değeri (ilkel değerler, nesneler, diziler, fonksiyonlar, hatalar, semboller) temsil etmek için kullanılabilen özel bir türdür (evrensel üst tür). Genellikle bir değerin türünün derleme zamanında bilinmediği durumlarda ya da TypeScript tür tanımları bulunmayan harici API'ler veya kütüphanelerden gelen değerlerle çalışırken kullanılır.

`any` türünü kullanarak TypeScript derleyicisine değerlerin herhangi bir sınırlama olmadan temsil edilmesi gerektiğini belirtirsiniz. Kodunuzdaki tür güvenliğini en üst düzeye çıkarmak için şunları göz önünde bulundurun:

- `any` kullanımını, türün gerçekten bilinmediği belirli durumlarla sınırlayın.
- Bir fonksiyondan `any` türü döndürmeyin; çünkü bu, onu kullanan kodun tür güvenliğini zayıflatır.
- Derleyiciyi susturmanız gerekiyorsa `any` yerine `@ts-ignore` kullanın.

```
let value: any;
value = true; // Valid
value = 7; // Valid
```

Unknown türü

TypeScript'te `unknown` türü, türü bilinmeyen bir değeri temsil eder. Herhangi bir türde değere izin veren `any` türünün aksine `unknown`, belirli bir şekilde kullanılmadan önce tür kontrolü veya tür onaylaması gerektirir; dolayısıyla önce tür onaylaması yapılmadan ya da daha belirli bir türe daraltılmadan bir `unknown` üzerinde hiçbir işleme izin verilmez.

`unknown` türü yalnızca `any` ve yine `unknown` türüne atanabilir ve `any` için tür güvenli bir alternatiftir.

```
let value: unknown;

let value1: unknown = value; // Valid
let value2: any = value; // Valid
let value3: boolean = value; // Invalid
let value4: number = value; // Invalid

const add = (a: unknown, b: unknown): number | undefined =>
     typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Void türü

`void` türü, bir fonksiyonun değer döndürmediğini belirtmek için kullanılır.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

Never türü

`never` türü, hiçbir zaman ortaya çıkmayan değerleri temsil eder. Hiçbir zaman dönmeyen veya hata fırlatan fonksiyonları ya da ifadeleri belirtmek için kullanılır.

Örneğin, sonsuz bir döngü:

```
const infiniteLoop = (): never => {
    while (true) {
        // do something
    }
};
```

Hata fırlatma:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

`never` türü, tür güvenliğini sağlamada ve kodunuzdaki olası hataları yakalamada kullanışlıdır. Diğer türler ve kontrol akışı ifadeleriyle birlikte kullanıldığında TypeScript'in daha kesin türleri analiz etmesine ve çıkarsamasına yardımcı olur. Örneğin:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // move up
            break;
        case 'down':
            // move down
            break;
        default:
            const exhaustiveCheck: never = direction;
                        throw new Error(`Unhandled direction:
${exhaustiveCheck}`);
```

```
    }
};
```

Arayüz ve Tür

Yaygın Sözdizimi

TypeScript'te arayüzler, bir nesnenin sahip olması gereken özelliklerin veya metotların adlarını ve türlerini belirterek nesnelerin yapısını tanımlar. TypeScript'te arayüz tanımlamak için yaygın sözdizimi şöyledir:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

Tür tanımı için de benzer şekilde:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` veya `type TypeName`: Arayüzün adını tanımlar. `property1: Type1`: Arayüzün özelliklerini karşılık gelen türleriyle birlikte belirtir. Her biri noktalı virgülle ayrılan birden fazla özellik tanımlanabilir. `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: Arayüzün metotlarını belirtir. Metotlar; adları, ardından parantez içindeki parametre listesi

ve dönüş türüyle tanımlanır. Her biri noktalı virgülle ayrılan birden fazla metot tanımlanabilir.

Arayüz örneği:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Tür örneği:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

TypeScript'te türler, verilerin şeklini tanımlamak ve tür denetimini zorunlu kılmak için kullanılır. Belirli kullanım durumuna bağlı olarak TypeScript'te tür tanımlamak için birkaç yaygın sözdizimi vardır. İşte bazı örnekler:

Temel Türler

```
let myNumber: number = 123; // number type
let myBoolean: boolean = true; // boolean type
let myArray: string[] = ['a', 'b']; // array of strings
let myTuple: [string, number] = ['a', 123]; // tuple
```

Nesneler ve Arayüzler

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

Birleşim ve Kesişim Türleri

```
type MyType = string | number; // Union type
let myUnion: MyType = 'hello'; // Can be a string
myUnion = 123; // Or a number

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Intersection type
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Object with both name and age properties
```

Yerleşik İlkel Türler

TypeScript, değişkenleri, fonksiyon parametrelerini ve dönüş türlerini tanımlamak için kullanılabilecek çeşitli yerleşik ilkel türlere sahiptir:

- `number`: Tam sayılar ve kayan noktalı sayılar dâhil olmak üzere sayısal değerleri temsil eder.
- `string`: Metinsel verileri temsil eder.
- `boolean`: Doğru veya yanlış olabilen mantıksal değerleri temsil eder.
- `null`: Bir değerin yokluğunu temsil eder.
- `undefined`: Atanmamış veya tanımlanmamış bir değeri temsil eder.
- `symbol`: Benzersiz bir tanımlayıcıyı temsil eder. Semboller genellikle nesne özellikleri için anahtar olarak kullanılır.
- `bigint`: Keyfî hassasiyetli tam sayıları temsil eder.
- `any`: Dinamik veya bilinmeyen bir türü temsil eder. any türündeki değişkenler her türden değer tutabilir ve tür denetimini atlar.
- `void`: Herhangi bir türün yokluğunu temsil eder. Genellikle değer döndürmeyen fonksiyonların dönüş türü olarak kullanılır.

- `never`: Hiçbir zaman ortaya çıkmayan değerlerin türünü temsil eder. Genellikle hata fırlatan veya sonsuz döngüye giren fonksiyonların dönüş türü olarak kullanılır.

Yaygın Yerleşik JS Nesneleri

TypeScript, JavaScript'in bir üst kümesidir; yaygın olarak kullanılan tüm yerleşik JavaScript nesnelerini içerir. Bu nesnelerin kapsamlı bir listesini Mozilla Developer Network (MDN) dokümantasyon sitesinde bulabilirsiniz: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Yaygın olarak kullanılan bazı yerleşik JavaScript nesnelerinin listesi şöyledir:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set
- Promise
- Intl

Aşırı Yüklemeler

TypeScript'te fonksiyon aşırı yüklemeleri, tek bir fonksiyon adı için birden fazla fonksiyon imzası tanımlamanıza ve böylece birden fazla şekilde çağrılabilen fonksiyonlar oluşturmanıza olanak tanır. İşte bir örnek:

```
// Overloads
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implementation
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Valid
sayHi(['aa', 'bb']); // Valid
```

Bir `class` içinde fonksiyon aşırı yüklemelerinin kullanımına ilişkin başka bir örnek:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // overload
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;
```

```
    // implementation
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

Birleştirme ve Genişletme

Birleştirme ve genişletme, türler ve arayüzlerle çalışmaya ilişkin iki farklı kavramı ifade eder.

Birleştirme, aynı ada sahip birden fazla bildirimi tek bir tanımda bir araya getirmenize olanak tanır. Örneğin, aynı ada sahip bir arayüzü birden fazla kez tanımladığınızda:

```
interface X {
    a: string;
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
    b: 7,
};
```

Genişletme, yenilerini oluşturmak için mevcut türleri veya arayüzleri genişletme ya da bunlardan kalıtım alma yeteneğini ifade eder. Mevcut bir türün özgün tanımını değiştirmeden ona ek özellikler veya metotlar eklemeye yarayan bir mekanizmadır. Örnek:

```
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

Tür ve Arayüz Arasındaki Farklar

Bildirim birleştirme (genişletme):

Arayüzler bildirim birleştirmeyi destekler; bu, aynı ada sahip birden fazla arayüz tanımlayabileceğiniz ve TypeScript'in bunları birleştirilmiş özellikler ile metotlara sahip tek bir arayüzde birleştireceği anlamına gelir. Öte yandan türler bildirim birleştirmeyi desteklemez. Bu, özgün tanımları değiştirmeden veya eksik ya da hatalı türlere yama

uygulamadan ek işlevsellik katmak veya mevcut türleri özelleştirmek istediğinizde yararlı olabilir.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

Diğer türleri/arayüzleri genişletme:

Hem türler hem de arayüzler diğer türleri/arayüzleri genişletebilir, ancak sözdizimleri farklıdır. Arayüzlerde, diğer arayüzlerin özelliklerini ve metotlarını devralmak için `extends` anahtar sözcüğünü kullanırsınız. Ancak bir arayüz, birleşim türü gibi karmaşık bir türü genişletemez.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

Türlerde ise birden fazla türü tek bir türde birleştirmek (kesişim) için & operatörünü kullanırsınız.

```
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

Birleşim ve Kesişim Türleri:

Birleşim ve Kesişim Türlerini tanımlama konusunda türler daha esnektir. `type` anahtar sözcüğüyle `|` operatörünü kullanarak kolayca birleşim türleri ve `&` operatörünü kullanarak kesişim türleri oluşturabilirsiniz. Arayüzler de birleşim türlerini dolaylı olarak temsil edebilse de kesişim türleri için yerleşik desteğe sahip değildir.

```
type Department = 'dep-x' | 'dep-y'; // Union

type Person = {
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Intersection
```

Arayüzlerle örnek:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Union of interfaces
```

Sınıf

Sınıfın Yaygın Sözdizimi

TypeScript'te bir sınıf tanımlamak için `class` anahtar sözcüğü kullanılır. Aşağıda bir örnek görebilirsiniz:

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

`class` anahtar sözcüğü, "Person" adlı bir sınıf tanımlamak için kullanılır.

Sınıfın iki özel özelliği vardır: `string` türünde name ve `number` türünde age.

Oluşturucu, `constructor` anahtar sözcüğü kullanılarak tanımlanır. name ve age parametrelerini alır ve bunları karşılık gelen özelliklere atar.

Sınıf, bir karşılama mesajını günlüğe yazan sayHi adlı `public` bir metoda sahiptir.

TypeScript'te bir sınıf örneği oluşturmak için `new` anahtar sözcüğünü, ardından sınıf adını ve parantezleri `()` kullanabilirsiniz. Örneğin:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Output: Hello, my name is John Doe and I
am 25 years old.
```

Oluşturucu

Oluşturucular, bir sınıf örneği oluşturulduğunda nesnenin özelliklerini başlatmak için kullanılan özel metotlardır.

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
```

```
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

Aşağıdaki sözdizimini kullanarak bir oluşturucuyu aşırı yüklemek mümkündür:

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

TypeScript'te birden fazla oluşturucu aşırı yüklemesi tanımlamak mümkündür, ancak tüm aşırı yüklemelerle uyumlu olması gereken yalnızca tek bir uygulamanız olabilir; bu, isteğe bağlı bir parametre kullanılarak gerçekleştirilebilir.

```
class Person {
    name: string;
    age: number;
```

```
    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Özel ve Korumalı Oluşturucular

TypeScript'te oluşturucular, erişilebilirliklerini ve kullanımlarını kısıtlayan private veya protected olarak işaretlenebilir.

Özel Oluşturucular: Yalnızca sınıfın kendi içinden çağrılabilir. Özel oluşturucular genellikle tekil nesne kalıbını zorunlu kılmak veya örneklerin oluşturulmasını sınıf içindeki bir fabrika metoduyla sınırlandırmak istediğiniz senaryolarda kullanılır.

Korumalı Oluşturucular: Korumalı oluşturucular, doğrudan örneklenmemesi ancak alt sınıflar tarafından genişletilebilmesi

gereken bir temel sınıf oluşturmak istediğinizde kullanışlıdır.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Attempting to instantiate the base class directly will result in an error
// const baseObj = new BaseClass(); // Error: Constructor of class 'BaseClass' is protected.

// Create an instance of the derived class
const derivedObj = new DerivedClass(10);
```

Erişim Belirleyicileri

Erişim Belirleyicileri `private`, `protected` ve `public`, TypeScript sınıflarında özellikler ve metotlar gibi sınıf üyelerinin görünürlüğünü ve erişilebilirliğini denetlemek için kullanılır. Bu belirleyiciler, kapsüllemeyi zorunlu kılmak ve bir sınıfın iç durumuna erişme ve onu değiştirme sınırlarını belirlemek açısından çok önemlidir.

`private` belirleyicisi, sınıf üyesine erişimi yalnızca onu içeren sınıfla sınırlar.

`protected` belirleyicisi, sınıf üyesine onu içeren sınıf ve bu sınıftan türetilen sınıflar içinden erişilmesine izin verir.

`public` belirleyicisi, sınıf üyesine kısıtlanmamış erişim sağlar ve her yerden erişilmesine olanak tanır.

Get ve Set

Getter ve setter'lar, sınıf özellikleri için özel erişim ve değiştirme davranışı tanımlamanıza olanak tanıyan özel metotlardır. Bir nesnenin iç durumunu kapsüllemenize ve özelliklerin değerlerini alırken veya ayarlarken ek mantık sağlamanıza imkân verirler. TypeScript'te getter ve setter'lar sırasıyla `get` ve `set` anahtar sözcükleri kullanılarak tanımlanır. İşte bir örnek:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

Sınıflarda Otomatik Erişimciler

TypeScript 4.9 sürümü, yakında sunulacak bir ECMAScript özelliği olan otomatik erişimciler için destek ekler. Bunlar sınıf

özelliklerine benzer, ancak “accessor” anahtar sözcüğüyle bildirilir.

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

Otomatik erişimcilerin sözdizimsel şekeri kaldırılarak, erişilemeyen bir özellik üzerinde çalışan özel `get` ve `set` erişimcilerine dönüştürülürler.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

TypeScript’te `this` anahtar sözcüğü, bir sınıfın metotları veya oluşturucuları içindeki geçerli örneğini ifade eder. Sınıfın özelliklerine ve metotlarına kendi kapsamı içinden erişmenize ve bunları değiştirmenize olanak tanır. Bir nesnenin iç

durumuna kendi metotları içinden erişmek ve bu durumu işlemek için bir yol sağlar.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

Parametre Özellikleri

Parametre özellikleri, sınıf özelliklerini doğrudan oluşturucu parametreleri içinde bildirip başlatmanıza ve böylece tekrarlanan koddan kaçınmanıza olanak tanır. Örneğin:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
            // The "private" and "public" keywords in the
constructor
              // automatically declare and initialize the
corresponding class properties.
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
```

```
}
const person = new Person('Alice', 25);
person.introduce();
```

Soyut Sınıflar

Soyut Sınıflar, TypeScript'te temel olarak kalıtım için kullanılır. Alt sınıfların devralabileceği ortak özellikleri ve metotları tanımlamanın bir yolunu sağlarlar. Bu, ortak davranış tanımlamak ve alt sınıfların belirli metotları uygulamasını zorunlu kılmak istediğinizde yararlıdır. Ayrıca soyut temel sınıfın, alt sınıflar için paylaşılan bir arayüz ve ortak işlevsellik sağladığı bir sınıf hiyerarşisi oluşturmanıza olanak tanırlar.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Output: Whiskers meows.
```

Jeneriklerle

Jeneriklere sahip sınıflar, farklı türlerle çalışabilen yeniden kullanılabilir sınıflar tanımlamanıza olanak tanır.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

Dekoratörler

Dekoratörler; meta veri eklemek, davranışı değiştirmek, doğrulamak veya hedef öğenin işlevselliğini genişletmek için bir mekanizma sağlar. Bunlar çalışma zamanında yürütülen fonksiyonlardır. Bir bildirime birden fazla dekoratör uygulanabilir.

Dekoratörler deneysel özelliklerdir ve aşağıdaki örnekler yalnızca ES6 kullanan TypeScript 5 veya üzeri sürümlerle

uyumludur.

TypeScript'in 5'ten önceki sürümlerinde, `experimentalDecorators` özelliğinin `tsconfig.json` dosyanızda ayarlanması veya komut satırınızda `--experimentalDecorators` kullanılmasıyla etkinleştirilmeleri gerekir (ancak aşağıdaki örnek çalışmaz).

Dekoratörlerin yaygın kullanım alanlarından bazıları şunlardır:

- Özellik değişikliklerini izleme.
- Metot çağrılarını izleme.
- Ek özellikler veya metotlar ekleme.
- Çalışma zamanı doğrulaması.
- Otomatik serileştirme ve seriden çıkarma.
- Günlüğe kaydetme.
- Yetkilendirme ve kimlik doğrulama.
- Hatalara karşı koruma.

Not: 5. sürüm dekoratörleri parametrelerin dekore edilmesine izin vermez.

Dekoratör türleri:

Sınıf Dekoratörleri

Sınıf Dekoratörleri, özellikler veya metotlar eklemek ya da bir sınıfın örneklerini toplamak gibi yollarla mevcut bir sınıfı genişletmek için kullanışlıdır. Aşağıdaki örnekte, sınıfı metinsel bir gösterime dönüştüren bir `toString` metodu ekliyoruz.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
```

```
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}

@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Logs:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

Özellik Dekoratörü

Özellik dekoratörleri, başlangıç değerlerini değiştirmek gibi bir özelliğin davranışını değiştirmek için kullanışlıdır. Aşağıdaki kodda, bir özelliği her zaman büyük harfli olacak şekilde ayarlayan bir betik bulunur:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Logs: HELLO!
```

Metot Dekoratörü

Metot dekoratörleri, metotların davranışını değiştirmenize veya geliştirmenize olanak tanır. Aşağıda basit bir günlükleme örneği verilmiştir:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

       function replacementMethod(this: This, ...args: Args):
Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
```

```
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

new MyClass().sayHello();
```

Şunları günlüğe kaydeder:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Getter ve Setter Dekoratörleri

Getter ve setter dekoratörleri, sınıf erişimcilerinin davranışını değiştirmenize veya geliştirmenize olanak tanır. Örneğin, özellik atamalarını doğrulamak için kullanışlıdırlar. İşte basit bir getter dekoratörü örneği:

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
```

```
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}

class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Valid: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Throw: Invalid!
```

Dekoratör Meta Verisi

Dekoratör Meta Verisi, dekoratörlerin herhangi bir sınıfa meta veri uygulama ve bu meta veriyi kullanma sürecini basitleştirir. Dekoratörler, bağlam nesnesindeki hem ilkel değerler hem de nesneler için anahtar görevi görebilen yeni bir meta veri özelliğine erişebilir. Meta veri bilgilerine sınıfta `Symbol.metadata` aracılığıyla erişilebilir.

Meta veri, hata ayıklama, serileştirme veya dekoratörlerle bağımlılık enjeksiyonu gibi çeşitli amaçlarla kullanılabilir.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Simple polyfill

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
    | ClassMethodDecoratorContext; // Context contains property metadata: DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Set the metadata object with a primitive value
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Get metadata information

console.log(JSON.stringify(metadata)); //
{"bar":true,"baz":true,"foo":true}
```

Kalıtım

Kalıtım, bir sınıfın temel sınıf veya üst sınıf olarak bilinen başka bir sınıftan özellikleri ve metotları devralabildiği mekanizmayı ifade eder. Alt sınıf olarak da adlandırılan türetilmiş sınıf, yeni özellikler ve metotlar ekleyerek veya mevcut olanları geçersiz kılarak temel sınıfın işlevselliğini genişletebilir ve özelleştirebilir.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Create an instance of the base class
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Create an instance of the derived class
```

```
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript geleneksel anlamda çoklu kalıtımı desteklemez ve bunun yerine tek bir temel sınıftan kalıtıma izin verir. TypeScript birden fazla arayüzü destekler. Bir arayüz, bir nesnenin yapısı için bir sözleşme tanımlayabilir ve bir sınıf birden fazla arayüzü uygulayabilir. Bu, bir sınıfın birden fazla kaynaktan davranış ve yapı devralmasına olanak tanır.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

TypeScript'teki `class` anahtar sözcüğü, JavaScript'te olduğu gibi genellikle sözdizimsel şeker olarak adlandırılır. Sınıf tabanlı bir şekilde nesne oluşturmak ve nesnelerle çalışmak için daha alışıldık bir sözdizimi sunmak amacıyla ECMAScript 2015'te

(ES6) kullanıma sunulmuştur. Ancak JavaScript'in bir üst kümesi olan TypeScript'in nihayetinde, temelinde prototip tabanlı kalmaya devam eden JavaScript'e derlendiğini unutmamak önemlidir.

Statik Üyeler

TypeScript statik üyelere sahiptir. Bir sınıfın statik üyelerine erişmek için nesne oluşturmaya gerek kalmadan sınıf adını ve ardından bir nokta kullanabilirsiniz.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Özellik Başlatma

TypeScript'te bir sınıfın özelliklerini başlatmanın birkaç yolu vardır:

Satır içinde:

Aşağıdaki örnekte, bir sınıf örneği oluşturulduğunda bu başlangıç değerleri kullanılır.

```
class MyClass {
    property1: string = 'default value';
```

```
    property2: number = 42;
}
```

Oluşturucuda:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

Oluşturucu parametrelerini kullanarak:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
            // There is no need to assign the values to the
properties explicitly.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

Metot Aşırı Yükleme

Metot aşırı yükleme, bir sınıfın aynı ada ancak farklı parametre türlerine veya farklı sayıda parametreye sahip birden fazla metoda sahip olmasını sağlar. Bu, bir metodu iletilen argümanlara göre farklı şekillerde çağırmamıza olanak tanır.

```
class MyClass {
    add(a: number, b: number): number; // Overload signature 1
    add(a: string, b: string): string; // Overload signature 2

      add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}

const r = new MyClass();
console.log(r.add(10, 5)); // Logs 15
```

Jenerikler

Jenerikler, birden çok türle çalışabilen yeniden kullanılabilir bileşenler ve işlevler oluşturmanıza olanak tanır. Jeneriklerle türleri, işlevleri ve arayüzleri parametreleştirerek bunların farklı türler üzerinde, söz konusu türleri önceden açıkça belirtmeden çalışmasını sağlayabilirsiniz.

Jenerikler, kodu daha esnek ve yeniden kullanılabilir hâle getirmenize olanak tanır.

Jenerik Tür

Bir jenerik tür tanımlamak için, tür parametrelerini belirtmek üzere açılı ayraçları (<>) kullanırsınız. Örneğin:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

Jenerik Sınıflar

Jenerikler sınıflara da uygulanabilir; böylece sınıflar, tür parametrelerini kullanarak birden çok türle çalışabilir. Bu, tür güvenliğini korurken farklı veri türleri üzerinde çalışabilen, yeniden kullanılabilir sınıf tanımları oluşturmak için kullanışlıdır.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

Jenerik Kısıtlamalar

Jenerik parametreler, `extends` anahtar sözcüğünün ardından tür parametresinin karşılaması gereken bir tür veya arayüz getirilerek kısıtlanabilir.

Aşağıdaki örnekte `T`'nin geçerli olabilmesi için uygun şekilde türü belirlenmiş bir `length` özelliğine sahip olması gerekir:

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Invalid
```

3.4 RC sürümünde sunulan dikkate değer bir jenerik özelliği, jenerik tür bağımsız değişkenlerini yayan yüksek dereceli işlev türü çıkarımıdır:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Bu işlevsellik, işlevsel programlamada yaygın olan tür güvenli point-free tarzda programlamayı kolaylaştırır.

Jeneriklerde Bağlamsal Daraltma

Jeneriklerde bağlamsal daraltma, TypeScript'te derleyicinin bir jenerik parametrenin türünü kullanıldığı bağlama göre daraltmasına olanak tanıyan mekanizmadır. Koşullu ifadelerde jenerik türlerle çalışırken kullanışlıdır:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Value is narrowed down to type 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Value is narrowed down to type 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

Silinen Yapısal Türler

TypeScript'te nesnelerin belirli ve tam bir türle eşleşmesi gerekmez. Örneğin, bir arayüzün gereksinimlerini karşılayan bir nesne oluşturursak, aralarında açık bir bağlantı kurulmamış olsa bile, bu nesneyi söz konusu arayüzün gerekli olduğu yerlerde kullanabiliriz. Örnek:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
```

```
    prop1: 'Origin',
};

log(obj); // Valid
```

Ad Alanları

TypeScript'te ad alanları, kodu mantıksal kapsayıcılar içinde düzenlemek, ad çakışmalarını önlemek ve ilişkili kodları bir arada gruplandırmak için kullanılır. `export` anahtar sözcüğünün kullanılması, modüllerin dışından ad alanına erişilmesine olanak tanır.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

Semboller

Semboller, programın yaşam süresi boyunca program genelinde benzersiz olması garanti edilen, değiştirilemez bir değeri temsil eden ilkel bir veri türüdür.

Semboller, nesne özellikleri için anahtar olarak kullanılabilir ve numaralandırılamayan özellikler oluşturmanın bir yolunu sağlar.

```ts
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

Artık WeakMap ve WeakSet'lerde sembollerin anahtar olarak kullanılmasına izin verilir.

Üç Eğik Çizgi Yönergeleri

Üç eğik çizgi yönergeleri, bir dosyanın nasıl işleneceği hakkında derleyiciye talimatlar veren özel yorumlardır. Bu yönergeler art arda üç eğik çizgiyle (`///`) başlar, genellikle bir TypeScript dosyasının en üstüne yerleştirilir ve çalışma zamanı davranışını etkilemez.

Üç eğik çizgi yönergeleri; harici bağımlılıklara başvurmak, modül yükleme davranışını belirtmek, belirli derleyici özelliklerini etkinleştirmek veya devre dışı bırakmak ve daha fazlası için kullanılır. Birkaç örnek:

Bir bildirim dosyasına başvurma:

```ts
/// <reference path="path/to/declaration/file.d.ts" />
```

Modül biçimini belirtme:

```ts
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Derleyici seçeneklerini etkinleştirme; aşağıdaki örnekte katı mod:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Tür İşleme

Türlerden Tür Oluşturma

Mevcut türleri birleştirerek, işleyerek veya dönüştürerek yeni türler oluşturmak mümkündür.

Kesişim Türleri (&):

Birden çok türü tek bir türde birleştirmenize olanak tanır:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Intersection of A and B
const obj: C = { foo: 42, bar: 'hello' };
```

Birleşim Türleri (|):

Birden fazla türden biri olabilen bir tür tanımlamanıza olanak tanır:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Eşlenmiş Türler:

Yeni bir tür oluşturmak için mevcut bir türün özelliklerini dönüştürmenize olanak tanır:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Properties become
read-only
```

Koşullu Türler:

Belirli koşullara göre türler oluşturmanıza olanak tanır:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P :
never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

İndeksli Erişim Türleri

TypeScript'te, `Type[Key]` indeksini kullanarak başka bir tür içindeki özelliklerin türlerine erişmek ve bunları işlemek mümkündür.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

Yardımcı Türler

Türleri işlemek için çeşitli yerleşik yardımcı türler kullanılabilir. En yaygın kullanılanların listesi aşağıdadır:

Awaited<T>

Promise türlerini özyinelemeli olarak açan bir tür oluşturur.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

T'nin tüm özellikleri isteğe bağlı olarak ayarlanmış bir tür oluşturur.

```
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?:
number | undefined; }
```

Required<T>

T'nin tüm özellikleri zorunlu olarak ayarlanmış bir tür oluşturur.

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

T'nin tüm özellikleri salt okunur olarak ayarlanmış bir tür oluşturur.

```
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Invalid
```

Record<K, T>

K özellik kümesindeki her özelliği T türünde olan bir tür oluşturur.

```
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

T'den belirtilen K özelliklerini seçerek bir tür oluşturur.

```
type Product = {
    name: string;
```

```
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

T'den belirtilen K özelliklerini çıkararak bir tür oluşturur.

```
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

U türündeki tüm değerleri T'den hariç tutarak bir tür oluşturur.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

T'den U türündeki tüm değerleri ayıklayarak bir tür oluşturur.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

T'den null ve undefined değerlerini hariç tutarak bir tür oluşturur.

```ts
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

T işlev türünün parametre türlerini elde eder.

```ts
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

T oluşturucu işlev türünün parametre türlerini elde eder.

```ts
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

T işlev türünün dönüş türünü elde eder.

```ts
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

T sınıf türünün örnek türünü elde eder.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

T işlev türündeki 'this' parametresinin türünü elde eder.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type  PersonThisType  =  ThisParameterType<Person['greet']>;  //
Person
```

OmitThisParameter<T>

T işlev türünden 'this' parametresini kaldırır.

```
function capitalize(this: String) {
                        return      this[0].toUpperCase()      +
this.substring(1).toLowerCase();
}
```

```
type CapitalizeType = OmitThisParameter<typeof capitalize>; // () => string
```

ThisType<T>

Bağlamsal bir `this` türü için işaretleyici görevi görür.

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } & ThisType<Logger> = {
    hello: function () {
        this.log('some error'); // Valid as "log" is a part of "this".
        this.update(); // Invalid
    },
};
```

Uppercase<T>

T girdi türünün adını büyük harfe dönüştürür.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

T girdi türünün adını küçük harfe dönüştürür.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

T girdi türünün adının ilk harfini büyütür.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

T girdi türünün adının ilk harfini küçültür.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer, bir jenerik işlevin kapsamında otomatik tür çıkarımını engellemek için tasarlanmış bir yardımcı türdür.

Örnek:

```
// Automatic inference of types within the scope of a generic
function.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Type here is ("a" | "b" |
"c")[]
```

NoInfer ile:

```
// Example function that uses NoInfer to prevent type inference
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Error: Type Argument of
type '"c"' is not assignable to parameter of type '"a" | "b"'.
```

Diğer Konular

Hatalar ve İstisna İşleme

TypeScript, standart JavaScript hata işleme mekanizmalarını kullanarak hataları yakalamanıza ve işlemenize olanak tanır:

Try-Catch-Finally Blokları:

```
try {
    // Code that might throw an error
} catch (error) {
    // Handle the error
} finally {
    // Code that always executes, finally is optional
}
```

Farklı hata türlerini de işleyebilirsiniz:

```
try {
    // Code that might throw different types of errors
} catch (error) {
    if (error instanceof TypeError) {
        // Handle TypeError
    } else if (error instanceof RangeError) {
        // Handle RangeError
    } else {
        // Handle other errors
    }
}
```

Özel Hata Türleri:

Error `class`'ını genişleterek daha özel hatalar belirtmek mümkündür:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}
```

```
throw new CustomError('This is a custom error.');
```

Mixin Sınıfları

Mixin sınıfları, birden çok sınıfın davranışını tek bir sınıfta bir araya getirip birleştirmenize olanak tanır. Derin kalıtım zincirlerine gerek kalmadan işlevselliği yeniden kullanmanın ve genişletmenin bir yolunu sağlarlar.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Extend MyClass to include the behavior of Identifiable and
Selectable
interface MyClass extends Identifiable, Selectable {}

// Function to apply mixins to a class
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {
```

```
Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                  Object.defineProperty(source.prototype, name,
descriptor);
            }
        });
    });
}

// Apply the mixins to MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

Eşzamansız Dil Özellikleri

TypeScript, JavaScript'in bir üst kümesi olduğundan JavaScript'in şu yerleşik eşzamansız dil özelliklerine sahiptir:

Promise'ler:

Promise'ler, başarı ve hata koşullarını işlemek için `.then()` ve `.catch()` gibi yöntemleri kullanarak eşzamansız işlemleri ve bunların sonuçlarını işlemenin bir yoludur.

Daha fazla bilgi için: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Async/await anahtar sözcükleri, Promise'lerle çalışmak için daha eşzamanlı görünen bir sözdizimi sağlamanın bir yoludur. `async` anahtar sözcüğü eşzamansız bir işlev tanımlamak için, `await` anahtar sözcüğü ise bir Promise çözümlenene veya reddedilene kadar yürütmeyi duraklatmak için async işlev içinde kullanılır.

Daha fazla bilgi için: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

Aşağıdaki API'ler TypeScript'te iyi desteklenir:

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Worker'lar: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Worker'lar: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Yineleyiciler ve Üreteçler

Hem yineleyiciler hem de üreteçler TypeScript'te iyi desteklenir.

Yineleyiciler, yineleyici protokolünü uygulayan ve bir koleksiyonun veya dizinin öğelerine tek tek erişmenin bir yolunu sağlayan nesnelerdir. Yinelemedeki bir sonraki öğeyi

işaret eden bir işaretçi içeren yapılardır. Bir `next()` yöntemine sahiptirler; bu yöntem dizideki bir sonraki değeri ve dizinin `done` olup olmadığını belirten bir boolean değeri döndürür.

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Üreteçler, yineleyicilerin oluşturulmasını basitleştiren ve `function*` sözdizimi kullanılarak tanımlanan özel işlevlerdir.

Değer dizisini tanımlamak için `yield` anahtar sözcüğünü kullanırlar ve değerler istendiğinde yürütmeyi otomatik olarak duraklatıp sürdürürler.

Üreteçler, yineleyici oluşturmayı kolaylaştırır ve özellikle büyük veya sonsuz dizilerle çalışmak için kullanışlıdır.

Örnek:

```
function* numberGenerator(start: number, end: number): Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript ayrıca eşzamansız yineleyicileri ve eşzamansız üreteçleri destekler.

Daha fazla bilgi için:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

TsDocs JSDoc Referansı

Bir JavaScript kod tabanıyla çalışırken tür bilgisi sağlayan ek açıklamalara sahip JSDoc yorumlarını kullanarak TypeScript'in doğru türü çıkarmasına yardımcı olmak mümkündür.

Örnek:

```
/**
 * Computes the power of a given number
 * @constructor
 * @param {number} base - The base value of the expression
  * @param {number} exponent - The exponent value of the
expression
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent:
number): number
```

Belgelerin tamamı bu bağlantıda sunulmaktadır: https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

3.7 sürümünden itibaren JavaScript JSDoc sözdiziminden .d.ts tür tanımları oluşturmak mümkündür. Daha fazla bilgiye buradan ulaşabilirsiniz: https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

@types kuruluşu altındaki paketler, mevcut JavaScript kütüphaneleri veya modülleri için tür tanımları sağlamak amacıyla kullanılan özel bir paket adlandırma kuralını izler. Örneğin şu komut:

```
npm install --save-dev @types/lodash
```

geçerli projenize `lodash` tür tanımlarını yükler.

Bir `@types` paketinin tür tanımlarına katkıda bulunmak için lütfen https://github.com/DefinitelyTyped/DefinitelyTyped adresine bir pull request gönderin.

JSX

JSX (JavaScript XML), JavaScript veya TypeScript dosyalarınız içinde HTML benzeri kod yazmanıza olanak tanıyan bir JavaScript dili sözdizimi uzantısıdır. HTML yapısını tanımlamak için React'te yaygın olarak kullanılır.

TypeScript, tür onaylaması ve statik analiz sağlayarak JSX'in yeteneklerini genişletir.

JSX kullanmak için `jsx` derleyici seçeneğini `tsconfig.json` dosyanızda ayarlamanız gerekir. Yaygın iki yapılandırma seçeneği:

- "preserve": JSX'i değiştirmeden .jsx dosyaları üretir. Bu seçenek, TypeScript'e JSX sözdizimini olduğu gibi tutmasını ve derleme işlemi sırasında dönüştürmemesini söyler. Dönüşümü gerçekleştiren Babel gibi ayrı bir aracınız varsa bu seçeneği kullanabilirsiniz.
- "react": TypeScript'in yerleşik JSX dönüşümünü etkinleştirir. React.createElement kullanılır.

Tüm seçeneklere buradan ulaşabilirsiniz: https://www.typescriptlang.org/tsconfig#jsx

ES6 Modülleri

TypeScript, ES6'yı (ECMAScript 2015) ve sonraki birçok sürümü destekler. Bu, ok işlevleri, şablon değişmezleri, sınıflar, modüller, yapı bozma ve daha fazlası gibi ES6 sözdizimini kullanabileceğiniz anlamına gelir.

Projenizde ES6 özelliklerini etkinleştirmek için tsconfig.json dosyasındaki `target` özelliğini belirtebilirsiniz.

Bir yapılandırma örneği:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

ES7 Üs Alma Operatörü

Üs alma (`**`) operatörü, ilk işlenenin ikinci işlenenin kuvvetine yükseltilmesiyle elde edilen değeri hesaplar. `Math.pow()` işlevine benzer şekilde çalışır, ancak buna ek olarak BigInt'leri işlenen olarak kabul edebilir. TypeScript, tsconfig.json dosyanızda `target` değerini `es2016` veya daha yüksek bir sürüme ayarladığınızda bu operatörü tam olarak destekler.

```
console.log(2 ** (2 ** 2)); // 16
```

for-await-of İfadesi

Bu, TypeScript tarafından tam olarak desteklenen ve `es2018` hedef sürümüyle eşzamansız yinelenebilir nesneler üzerinde yineleme yapmanıza olanak tanıyan bir JavaScript özelliğidir.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

Yeni target Meta Özelliği

TypeScript'te `new.target` meta özelliğini kullanabilirsiniz. Bu özellik, bir işlevin veya oluşturucunun new operatörü kullanılarak çağrılıp çağrılmadığını belirlemenizi sağlar. Bir nesnenin oluşturucu çağrısı sonucunda oluşturulup oluşturulmadığını saptamanıza olanak tanır.

```
class Parent {
    constructor() {
            console.log(new.target);  // Logs the constructor
function used to create an instance
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}
```

```
const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Dinamik İçe Aktarma İfadeleri

TypeScript tarafından desteklenen dinamik içe aktarmaya yönelik ECMAScript önerisini kullanarak modülleri koşullu olarak veya isteğe bağlı biçimde gecikmeli yüklemek mümkündür.

TypeScript'te dinamik içe aktarma ifadelerinin sözdizimi şöyledir:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
            const widget = await import('./widget'); // Dynamic
import
        widget.render(container);
    }
}

renderWidget();
```

"tsc –watch"

Bu komut, TypeScript dosyaları her değiştirildiğinde bunları otomatik olarak yeniden derleyebilme özelliğiyle TypeScript derleyicisini `--watch` parametresiyle başlatır.

```
tsc --watch
```

TypeScript 4.9 sürümünden itibaren dosya izleme öncelikle dosya sistemi olaylarına dayanır; olay tabanlı bir izleyici kurulamazsa otomatik olarak yoklamaya başvurur.

Null Olmayan Onaylama Operatörü

Kesin atama onaylamaları olarak da adlandırılan null olmayan onaylama operatörü (sonek !), TypeScript'in statik tür analizi bir değişkenin veya özelliğin null ya da undefined olabileceğini öne sürse bile bunun null veya undefined olmadığını onaylamanıza olanak tanıyan bir TypeScript özelliğidir. Bu özellik sayesinde açık kontrolleri kaldırmak mümkündür.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

Varsayılan Değerli Bildirimler

Varsayılan değerli bildirimler, bir değişkene veya parametreye varsayılan bir değer atandığında kullanılır. Bu, söz konusu değişken veya parametre için bir değer sağlanmazsa bunun yerine varsayılan değerin kullanılacağı anlamına gelir.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

İsteğe Bağlı Zincirleme

İsteğe bağlı zincirleme operatörü `?.`, özelliklere veya yöntemlere erişmek için normal nokta operatörü (`.`) gibi çalışır. Ancak null veya undefined değerlerini, hata oluşturmak yerine

ifadeyi sonlandırıp `undefined` döndürerek sorunsuz biçimde işler.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

Nullish Birleştirme Operatörü

Nullish birleştirme operatörü `??`, sol taraf `null` veya `undefined` ise sağ taraftaki değeri; aksi takdirde sol taraftaki değeri döndürür.

```
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

Şablon Değişmez Türleri

Şablon Değişmez Türleri, string değerlerini tür düzeyinde işlemenize ve mevcut türlerden yeni string türleri

oluşturmanıza olanak tanır. String tabanlı işlemlerden daha anlamlı ve kesin türler oluşturmak için kullanışlıdır.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

İşlev Aşırı Yükleme

İşlev aşırı yükleme, aynı işlev adı için her biri farklı parametre ve dönüş türlerine sahip birden çok işlev imzası tanımlamanıza olanak tanır. Aşırı yüklenmiş bir işlevi çağırdığınızda TypeScript, doğru işlev imzasını belirlemek için sağlanan bağımsız değişkenleri kullanır:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Özyinelemeli Türler

Özyinelemeli Tür, kendisine başvurabilen bir türdür. Bağlı listeler, ağaçlar ve graflar gibi hiyerarşik veya özyinelemeli bir

yapıya (potansiyel olarak sonsuz iç içe geçmeye) sahip veri yapılarını tanımlamak için kullanışlıdır.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Özyinelemeli Koşullu Türler

TypeScript'te mantık ve özyineleme kullanarak karmaşık tür ilişkileri tanımlamak mümkündür. Bunu basit ifadelerle ele alalım:

Koşullu Türler, mantıksal koşullara göre türler tanımlamanıza olanak tanır:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

Özyineleme, kendi tanımı içinde kendisine başvuran bir tür tanımı anlamına gelir:

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
    },
};
```

Özyinelemeli Koşullu Türler, hem koşullu mantığı hem de özyinelemeyi birleştirir. Bu, bir tür tanımının koşullu mantık aracılığıyla kendisine bağlı olabileceği ve böylece karmaşık ve esnek tür ilişkileri oluşturabileceği anlamına gelir.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 |
1 | 6
```

Node'da ECMAScript Modülü Desteği

Node.js, 15.3.0 sürümünden itibaren ECMAScript Modülleri desteğini ekledi ve TypeScript, 4.7 sürümünden beri Node.js için ECMAScript Modülü Desteğine sahiptir. Bu destek, tsconfig.json dosyasındaki `module` özelliği `nodenext` değerine ayarlanarak etkinleştirilebilir. İşte bir örnek:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js, modüller için iki dosya uzantısını destekler: ES modülleri için `.mjs` ve CommonJS modülleri için `.cjs`. TypeScript'teki eşdeğer dosya uzantıları, ES modülleri için `.mts` ve CommonJS modülleri için `.cts` şeklindedir. TypeScript derleyicisi bu dosyaları JavaScript'e dönüştürdüğünde `.mjs` ve `.cjs` dosyaları oluşturur.

Projenizde ES modüllerini kullanmak istiyorsanız package.json dosyasındaki `type` özelliğini "module" olarak ayarlayabilirsiniz. Bu, Node.js'e projeyi bir ES modülü projesi olarak ele alması talimatını verir.

Ayrıca TypeScript, .d.ts dosyalarındaki tür bildirimlerini de destekler. Bu bildirim dosyaları, TypeScript ile yazılmış kütüphaneler veya modüller için tür bilgisi sağlayarak diğer geliştiricilerin bunları TypeScript'in tür denetimi ve otomatik tamamlama özellikleriyle kullanabilmesine olanak tanır.

Onaylama Fonksiyonları

TypeScript'te onaylama fonksiyonları, dönüş değerlerine göre belirli bir koşulun doğrulandığını belirten fonksiyonlardır. En basit biçimiyle bir onaylama fonksiyonu, sağlanan bir yüklemi inceler ve yüklem false olarak değerlendirildiğinde hata oluşturur.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

Ya da bir fonksiyon ifadesi olarak bildirilebilir:

```
type AssertIsNumber = (value: unknown) => asserts value is
number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

Onaylama fonksiyonları, tür koruyucularıyla benzerlik gösterir. Tür koruyucuları başlangıçta çalışma zamanı denetimleri gerçekleştirmek ve bir değerin belirli bir kapsam içindeki türünü güvence altına almak için kullanıma sunulmuştur. Daha açık bir ifadeyle tür koruyucu, bir tür yüklemini değerlendiren ve yüklemin doğru mu yanlış mı olduğunu belirten bir boolean değeri döndüren fonksiyondur. Bu, yüklem karşılanmadığında false döndürmek yerine hata oluşturmayı amaçlayan onaylama fonksiyonlarından biraz farklıdır.

Tür koruyucu örneği:

```
const isNumber = (value: unknown): value is number => typeof
value === 'number';
```

Değişken Sayıda Öğeli Demet Türleri

Değişken Sayıda Öğeli Demet Türleri, TypeScript 4.0 sürümünde kullanıma sunulan bir özelliktir; bu nedenle önce demetin ne olduğunu yeniden ele alalım:

Demet türü, uzunluğu tanımlı ve her bir öğesinin türü bilinen bir dizidir:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

“Variadic” terimi, belirsiz sayıda argüman alma (değişken sayıda argüman kabul etme) anlamına gelir.

Değişken sayıda öğeli demet, önceki tüm özelliklere sahip olan ancak kesin yapısı henüz tanımlanmamış bir demet türüdür:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];
```

```
type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

Önceki kodda, demet yapısının aktarılan `T` jeneriği tarafından tanımlandığını görebiliriz.

Değişken sayıda öğeli demetler birden fazla jenerik kabul edebilir, bu da onları oldukça esnek kılar:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

Yeni değişken sayıda öğeli demetlerle şunları kullanabiliriz:

- Demet türü sözdizimindeki yayma ifadeleri artık jenerik olabilir; böylece üzerinde işlem yaptığımız gerçek türleri bilmediğimizde bile demetler ve diziler üzerindeki yüksek dereceli işlemleri temsil edebiliriz.
- Rest öğeleri bir demetin herhangi bir yerinde bulunabilir.

Örnek:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Kutulanmış Türler

Kutulanmış türler, ilkel türleri nesne olarak temsil etmek için kullanılan sarmalayıcı nesneleri ifade eder. Bu sarmalayıcı nesneler, doğrudan ilkel değerlerde bulunmayan ek işlevler ve metotlar sağlar.

`charAt` veya `normalize` gibi bir metot bir `string` ilkel değeri üzerinde çağrıldığında JavaScript, değeri bir `String` nesnesiyle sarmalar, metodu çağırır ve ardından nesneyi atar.

Gösterim:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript, ilkel değerler ve bunlara karşılık gelen nesne sarmalayıcıları için ayrı türler sağlayarak bu ayrımı temsil eder:

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

Kutulanmış türlere genellikle gerek yoktur. Kutulanmış türleri kullanmaktan kaçının ve bunun yerine ilkel türleri kullanın; örneğin `string` kullanın, `String` değil.

TypeScript'te Kovaryans ve Kontravaryans

Kovaryans ve kontravaryans, jenerik türlerdeki tür ilişkilerini açıklar. `Dog` bir `Animal` türüdür, ancak her `Animal` bir `Dog` değildir.

Kovaryans (değer üretme): `Dog` döndüren bir fonksiyon, `Animal` döndüren bir fonksiyonun yerine kullanılabilir. Her köpek bir hayvandır, ancak herhangi bir hayvan döndüren fonksiyon kedi döndürebilir.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // Valid
produceDog = produceAnimal; // Error
```

Kontravaryans (değer kabul etme): herhangi bir `Animal` kabul eden fonksiyon, `Dog` kabul eden fonksiyonun yerine kullanılabilir. Köpekleri de işleyebilir; yalnızca köpek kabul eden fonksiyon tüm hayvanları güvenle işleyemez.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Valid
consumeAnimal = consumeDog; // Error with strictFunctionTypes
```

Oklar güvenli atamaların yönünü gösterir:

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

Değiştirilebilir diziler: TypeScript, `Dog[]` değerinin `Animal[]` türüne atanmasına da izin verir. İki değişken aynı diziyi gösterir; daha geniş tür üzerinden köpek olmayan bir `Animal` eklenebilir.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Allowed, but unsafe
// dogs now contains an Animal that is not a Dog.
```

Derleyici seçeneği: `strictFunctionTypes` etkinse bağımsız fonksiyon parametreleri kontravaryant olarak denetlenir. Etkin değilse TypeScript güvenli olmasa da iki yönü kabul edebilir (bivaryans). Metot ve kurucu bildirimlerindeki parametreler bu denetimin istisnasıdır.

Tür Parametreleri için İsteğe Bağlı Varyans Ek Açıklamaları

TypeScript 4.7'den itibaren `out` kovaryansı, `in` kontravaryansı ve `in out` invaryansı belirtir. TypeScript genellikle varyansı çıkarır. Ek açıklamalar jenerik parametrenin kullanımına uygun olmalı, davranışı keyfî olarak değiştirmemelidir.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

Şablon Dizesi Kalıbı İndeks İmzaları

Şablon dizesi kalıbı indeks imzaları, şablon dizesi kalıplarını kullanarak esnek indeks imzaları tanımlamamıza olanak tanır. Bu özellik, belirli dize anahtarı kalıplarıyla indekslenebilen nesneler oluşturmamızı sağlayarak özelliklere erişirken ve özellikleri değiştirirken daha fazla denetim ve kesinlik sunar.

TypeScript, 4.4 sürümünden itibaren semboller ve şablon dizesi kalıpları için indeks imzalarına izin verir.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Unique symbol key
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

satisfies Operatörü

`satisfies` operatörü, belirli bir türün belirli bir arayüzü veya koşulu karşılayıp karşılamadığını denetlemenize olanak tanır. Başka bir deyişle bir türün, belirli bir arayüzün gerekli tüm özelliklerine ve metotlarına sahip olmasını sağlar. Bir

değişkenin bir tür tanımına uyduğundan emin olmanın bir yoludur. İşte bir örnek:

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Type Annotation using `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// In the following lines, TypeScript won't be able to infer properly
user.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
user.nickName; // string | string[] | undefined

// Type assertion using `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Here too, TypeScript won't be able to infer properly
user2.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
user2.nickName; // string | string[] | undefined

// Using the `satisfies` operator we can properly infer the types now
const user3 = {
    name: 'Simon',
    nickName: undefined,
```

```
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log);    // TypeScript infers
correctly: string[]
user3.nickName; // TypeScript infers correctly: undefined
```

Yalnızca Tür İçe Aktarımları ve Dışa Aktarımı

Yalnızca Tür İçe Aktarımları ve Dışa Aktarımları, bu türlerle ilişkili değerleri veya fonksiyonları içe ya da dışa aktarmadan türleri içe veya dışa aktarmanıza olanak tanır. Bu, paketinizin boyutunu küçültmek için yararlı olabilir.

Yalnızca tür içe aktarımlarını kullanmak için `import type` anahtar sözcüğünü kullanabilirsiniz.

TypeScript, `allowImportingTsExtensions` ayarlarından bağımsız olarak yalnızca tür içe aktarımlarında hem bildirim hem de uygulama dosyası uzantılarının (.ts, .mts, .cts ve .tsx) kullanılmasına izin verir.

Örneğin:

```
import type { House } from './house.ts';
```

Aşağıdaki biçimler desteklenir:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

using Bildirimi ve Belirtik Kaynak Yönetimi

Bir `using` bildirimi, elden çıkarılabilir kaynakları yönetmek için kullanılan ve `const` benzeri, blok kapsamlı, değişmez bir bağlamadır. Bir değerle başlatıldığında bu değerin `Symbol.dispose` metodu kaydedilir ve daha sonra çevreleyen blok kapsamından çıkıldığında çalıştırılır.

Bu, ECMAScript'in Kaynak Yönetimi özelliğine dayanır. Bu özellik; bağlantıları kapatma, dosyaları silme ve belleği serbest bırakma gibi nesne oluşturulduktan sonra yapılması gereken önemli temizleme görevleri için kullanışlıdır.

Notlar:

- TypeScript 5.2 sürümünde yakın zamanda kullanıma sunulduğu için çoğu çalışma zamanı yerel desteğe sahip değildir. Şunlar için polyfill kullanmanız gerekir: `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.
- Ayrıca tsconfig.json dosyanızı aşağıdaki gibi yapılandırmanız gerekir:

```
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

Örnek:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Simple polyfill

const doWork = (): Disposable => {
    return {
```

```
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Resource is declared
    console.log(2);
} // Resource is disposed (e.g., `work[Symbol.dispose]()` is
evaluated)

console.log(3);
```

Kod şunları günlüğe yazar:

```
1
2
disposed
3
```

Elden çıkarılmaya uygun bir kaynak, `Disposable` arayüzüne uymalıdır:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

`using` bildirimleri, kaynak elden çıkarma işlemlerini bir yığına kaydederek kaynakların bildirim sırasının tersine göre elden çıkarılmasını sağlar:

```
{
    using j = getA(),
        y = getB();
```

```
    using k = getC();
} // disposes `C`, then `B`, then `A`.
```

Daha sonraki kod çalışsa veya istisnalar oluşsa bile kaynakların elden çıkarılması garanti edilir. Bu durum, elden çıkarma işleminin istisna oluşturmasına ve başka bir istisnayı muhtemelen bastırmasına yol açabilir. Bastırılan hatalara ilişkin bilgileri korumak için `SuppressedError` adında yeni bir yerleşik istisna kullanıma sunulmuştur.

await using Bildirimi

Bir `await using` bildirimi, eşzamansız olarak elden çıkarılabilen bir kaynağı işler. Değerin, blok sona erdiğinde tamamlanması beklenecek bir `Symbol.asyncDispose` metodu olmalıdır.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Resource is declared
}     //     Resource     is     disposed     (e.g.,     `await
work[Symbol.asyncDispose]()` is evaluated)
```

Eşzamansız olarak elden çıkarılabilen bir kaynak, `Disposable` veya `AsyncDisposable` arayüzlerinden birine uymalıdır:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose   ??=   Symbol('Symbol.asyncDispose');   //
Simple polyfill

class DatabaseConnection implements AsyncDisposable {
     // A method that is called when the object is disposed
asynchronously
```

```
    [Symbol.asyncDispose]() {
        // Close the connection and return a promise
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
            await new Promise(resolve => setTimeout(resolve,
1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
     // Create a new connection and dispose it asynchronously
when it goes out of scope
      await using connection = new DatabaseConnection(); //
Resource is declared
    console.log('Doing some work...');
}     //     Resource     is     disposed     (e.g.,     `await
connection[Symbol.asyncDispose]()` is evaluated)

doWork();
```

Kod şunları günlüğe yazar:

```
Doing some work...
Closing the connection...
Connection closed.
```

`using` ve `await using` bildirimlerine şu deyimlerde izin verilir: `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

İçe Aktarma Nitelikleri

TypeScript 5.3'ün İçe Aktarma Nitelikleri (içe aktarımlar için etiketler), çalışma zamanına modüllerin (JSON vb.) nasıl işleneceğini bildirir. Bu, içe aktarımların açıkça tanımlanmasını

sağlayarak güvenliği artırır ve kaynakların daha güvenli yüklenmesi için İçerik Güvenliği Politikası (CSP) ile uyumludur. TypeScript bunların geçerli olduğundan emin olur, ancak belirli modül işleme yöntemlerine ilişkin yorumlamayı çalışma zamanına bırakır.

Örnek:

```
import config from './config.json' with { type: 'json' };
```

Dinamik içe aktarımla:

```
const config = import('./config.json', { with: { type: 'json' }
});
```

Düzenli İfade Sözdizimi Denetimi

TypeScript, 5.5.4 sürümünden itibaren düzenli ifade değişmezlerini derleme zamanında yaygın hatalara karşı denetler (ör. geçersiz sözdizimi, yanlış geriye başvurular, hedef JavaScript sürümünüz tarafından desteklenmeyen özellikler). Bu, hataları daha erken yakalamaya yardımcı olur, ancak new RegExp("...") dizelerini denetlemez.

```
let r = /(a)\2/; // Error: This backreference refers to a group
that does not exist.
```

import defer

`import defer`, bir modülü yüklemenize ancak siz gerçekten modülden bir şey kullanana kadar yürütülmesini ertelemenize olanak tanır. Bu, gereksiz işlemlerden ve yan etkilerden kaçınmaya yardımcı olur.

- Yalnızca şununla çalışır: `import defer * as name from "module"`
- Kod yalnızca dışa aktarılan bir öğeye eriştiğinizde çalışır